



دانشگاه صنعتی شریف
دانشکده مهندسی کامپیوتر
گزارش پروژه درس معماری کامپیوتر

عنوان:

تحلیل گلوگاه‌های سخت‌افزاری استنتاج مدل‌های زبانی بزرگ بر روی یک کارت گرافیک

نگارش

امیریوسف عبدی ۴۰۳۱۰۶۲۸۴

متین فدوی ۴۰۳۱۰۶۳۹۲

علی عرشیا محبی ۴۰۳۱۰۶۶۳۵

کیان قنبری ۴۰۳۱۰۶۴۸۷

استاد

دکتر اسدی

بهار ۱۴۰۵

فهرست مطالب

۳	۱ لینک گیتهاب پروژه
۳	۲ مقدمه
۴	۳ ابزارها و مفاهیم اولیه
۴	۱-۳ ابزارها
۴	۲-۳ مفاهیم و گلوگاه‌های عملکردی
۵	۳-۳ معیارهای ارزیابی عملکرد
۵	۴ گام‌ها
۵	۱-۴ گام ۱: اطمینان از عملکرد محیط
۶	۲-۴ گام ۲: جزئیات اجرای دستور vectorAdd
۶	۱-۲-۴ صدور فرامین توسط پردازنده مرکزی (CUDA API)
۸	۲-۲-۴ اجرای محاسبات درون هسته‌های گرافیکی (GPU Kernel)
۸	۳-۲-۴ انتقال داده‌ها در گذرگاه ارتباطی (GPU MemOps)
۱۰	۳-۴ گام ۳: اجرای مدل، پروفایلینگ سخت‌افزاری و ارزیابی عملکرد
۱۰	۱-۳-۴ مشخصات کلی مدل و تنظیمات پایه
۱۱	۲-۳-۴ دستور راه‌اندازی سرور و پروفایلینگ
۱۲	۳-۳-۴ دستور فشارسنجی و تولید بار پردازشی
۱۳	۴-۳-۴ مانیتور وضعیت CPU ، GPU ، RAM
۱۴	۵-۳-۴ معیارهای عملکردی (Performance Metrics)
۱۴	۶-۳-۴ معیارهای مانیتورینگ سخت‌افزار (Hardware Monitoring)
	۴-۴ گام ۴: پروفایلینگ و ارزیابی عملکرد مدل هنگام offload بخشی از آن به Main
۱۶	Memory

۵-۴	گام 5: تحلیل جامع گلوگاه‌ها و ارزیابی موازنه سیستم (Synthesis & Bottle neck Analysis)	۲۲
۵	قسمت امتیازی: بررسی تغییر اندازه Batch Size بر میزان استفاده از GPU، پهنای باند حافظه و عملکرد سیستم	۳۰
۱-۰-۵	راه‌اندازی vLLM	۳۰
۲-۰-۵	اجرای بنچمارک و سنجش کارایی (GuideLLM)	۳۱
۱-۵	کارایی (Performance)	۳۲
۱-۱-۵	توان عملیات (Throughput)	۳۲
۲-۱-۵	تاخیر (Latency)	۳۲
۳-۱-۵	نتایج به دست آمده	۳۶
۲-۵	پایش منابع سخت‌افزاری و تحلیل رفتار سیستم (Hardware Metrics & Re-source Utilization)	۳۶
۶	منابع (لینک‌های مفید)	۳۸

۱ لینک گیتهاب پروژه

<https://github.com/MatinFadavi0/llm-hardware-bottleneck-analysis>

۲ مقدمه

مدل‌های زبانی بزرگ امروزه به یکی از مهم‌ترین کاربردهای پردازش موازی و معماری پردازنده‌های گرافیکی تبدیل شده‌اند. اگرچه کارت‌های گرافیک مدرن از توان پردازشی بسیار بالایی برخوردارند، اما عملکرد استنتاج این مدل‌ها صرفاً به قدرت پردازشی خام محدود نمی‌شود. در بسیاری از کاربردهای عملی، گلوگاه عملکرد سیستم می‌تواند ناشی از محدودیت‌های پهنای باند حافظه، ظرفیت حافظه گرافیکی، نرخ انتقال داده از طریق گذرگاه PCIe، پردازنده مرکزی، حافظه اصلی و یا نحوه مدیریت حافظه توسط زمان‌اجرای CUDA باشد.

برای غلبه بر محدودیت حافظه گرافیکی، چارچوب‌های پیشرفته‌ای نظیر vLLM توسعه یافته‌اند که از قابلیت CPU Memory Offloading پشتیبانی می‌کنند. در این معماری، بخشی از پارامترهای مدل به جای حافظه کارت گرافیک، در حافظه اصلی سیستم مستقر شده و در زمان نیاز از طریق گذرگاه PCIe منتقل می‌شوند. اگرچه این قابلیت امکان اجرای مدل‌های بسیار بزرگ‌تر از ظرفیت فیزیکی کارت گرافیک را فراهم می‌آورد، اما به نوبه خود می‌تواند به ایجاد گلوگاه‌های سخت‌افزاری جدیدی منجر شود.

هدف اصلی از انجام این پروژه، تحلیل جامع عملکرد اجرای مدل‌های زبانی بزرگ بر روی یک سیستم تک‌کارت‌گرافیکی است. در این راستا، رفتار سخت‌افزاری سیستم در دو حالت «اجرای عادی» و «اجرای همراه با CPU Memory Offloading» پایش و ارزیابی می‌شود. با بهره‌گیری از ابزارهای دقیق پروفایلینگ و تحلیل عملکرد نظیر nvidia-smi، Nsight Systems، اطلاعات مرتبط با فراخوانی‌های CUDA، میزان درگیری پردازنده مرکزی و گرافیکی، وضعیت حافظه و انتقال داده‌ها جمع‌آوری می‌گردد. در نهایت، با تحلیل این داده‌ها مشخص خواهد شد که گلوگاه اصلی سیستم در هر سناریو به کدام بخش از معماری رایانه مربوط است.

۳ ابزارها و مفاهیم اولیه

۱-۳ ابزارها

- **vLLM**: یک کتابخانه و فریم‌ورک برای استنتاج و سرویس‌دهی مدل‌های زبانی بزرگ است که با تکنیک‌هایی نظیر PagedAttention و پشتیبانی از CPU Offloading، بهره‌وری حافظه و سرعت تولید توکن را به شکل چشمگیری افزایش می‌دهد.
- **Nsight Systems**: ابزار حرفه‌ای و جامع شرکت انویدیا برای پروفایلینگ سیستم است که دیدی دقیق از خط‌زمان (Timeline) اجرای برنامه‌ها شامل تحلیل هم‌زمان فعالیت‌های CPU، کرنل‌های CUDA و انتقال داده از طریق PCIe ارائه می‌دهد.
- **nvidia-smi**: یک ابزار خط فرمانی و کاربردی برای پایش لحظه‌ای و مدیریت وضعیت کارت گرافیک است که اطلاعاتی نظیر میزان درگیری GPU، مصرف حافظه و دمای سخت‌افزار را نمایش می‌دهد.

۲-۳ مفاهیم و گلوگاه‌های عملکردی

- **Compute-bound**: زمانی رخ می‌دهد که گلوگاه اصلی سیستم، توان محاسبه و تعداد عملیات ریاضی پردازنده (مانند FLOPs) باشد و نرخ انتقال داده مانعی برای اجرای سریع‌تر ایجاد نکند.
- **GPU Memory-bound**: زمانی ایجاد می‌شود که سرعت اجرای عملیات تحت شعاع پهنای باند یا ظرفیت حافظه اختصاصی کارت گرافیک (VRAM) قرار گیرد و پردازنده گرافیکی منتظر دریافت داده از حافظه بماند.
- **PCIe Bandwidth-bound**: حالتی که نرخ انتقال داده‌ها میان حافظه اصلی سیستم (RAM) و حافظه کارت گرافیک از طریق گذرگاه PCIe محدودکننده سرعت اجرای سیستم باشد.
- **CPU-bound**: هنگامی که پردازنده مرکزی سیستم به دلیل پردازش‌های سنگین غیرگرافیکی، زمان‌بندی فراخوانی‌ها یا آماده‌سازی داده‌ها نتواند کارت گرافیک را به طور کامل تغذیه کند و مانع اصلی سرعت باشد.

- **Main Memory-bound**: حالتی که پهنای باند یا سرعت دسترسی به حافظه اصلی سیستم (RAM)، به‌ویژه هنگام خواندن یا نوشتن داده‌ها در سناریوهای Offloading، تبدیل به گلوگاه اصلی شود.

۳-۳ معیارهای ارزیابی عملکرد

- **Time to First Token (TTFT)**: مدت زمانی که طول می‌کشد تا سیستم پس از دریافت ورودی، اولین توکن خروجی را تولید کند؛ این معیار نشان‌دهنده زمان تأخیر فاز پردازش ورودی (Prefill Phase) است.
- **Throughput**: نرخ کلی تولید توکن‌ها در واحد زمان (معمولاً توکن بر ثانیه) که نشان‌دهنده ظرفیت کلی و سرعت کلی سیستم در سرویس‌دهی است.
- **GPU Utilization**: درصد زمانی که هسته‌های پردازشی کارت گرافیک در حال انجام محاسبات فعال هستند.
- **GPU Memory Usage**: میزان حجم اشغال‌شده از حافظه اختصاصی کارت گرافیک (VRAM) توسط پارامترهای مدل، بافرها و KV Cache.
- **CPU Utilization**: درصد میزان درگیری و میزان بار کاری پردازنده مرکزی در طول فرایند اجرای مدل.
- **Main Memory Usage**: مقدار حافظه اصلی سیستم (RAM) که برای نگهداری داده‌ها، وزن‌های Offload - شده و بافرهای سیستم تخصیص یافته است.
- **CUDA Kernel Execution Time**: زمان دقیق صرف‌شده برای اجرای یک کرنل خاص CUDA بر روی کارت گرافیک که توسط ابزارهای پروفایلینگ اندازه‌گیری می‌شود.

۴ گام‌ها

۱-۴ گام ۱: اطمینان از عملکرد محیط

در این گام، هدف اصلی آماده‌سازی محیط نرم‌افزاری و اطمینان از عملکرد صحیح آن است. در ابتدا درایورهای NVIDIA و CUDA نصب گردید و برای اطمینان از صحت نصب محیط، یک مثال

ساده از CUDA به نام vectorAdd اجرا شد.

همان‌طور که در شکل ۱ مشاهده می‌شود، این برنامه با موفقیت اجرا شده است. خروجی ترمینال نشان می‌دهد که عملیات جمع برداری برای 50000 عنصر به درستی انجام شده است. همچنین، فرایند کپی کردن داده‌های ورودی از حافظه میزبان به دستگاه CUDA، اجرای کرنل با 196 بلوک شامل 256 رشته (threads) و کپی مجدد داده‌های خروجی به حافظه میزبان با موفقیت طی شده و پیغام Test PASSED در انتهای اجرا چاپ گردیده است. این خروجی تایید می‌کند که ارتباط میان پردازنده و کارت گرافیک برقرار بوده و محیط CUDA برای اجرای مدل‌های زبانی بزرگ آماده است.

```
• > ./step1_test/test_vectorAdd
[Vector addition of 50000 elements]
Copy input data from the host memory to the CUDA device
CUDA kernel launch with 196 blocks of 256 threads
Copy output data from the CUDA device to the host memory
Test PASSED
Done
```

شکل ۱: خروجی اجرای برنامه vectorAdd برای بررسی صحت نصب محیط CUDA

۲-۴ گام ۲: جزئیات اجرای دستور vectorAdd

در این گام با استفاده از ابزار تحلیل عملکرد Nsight Systems، به بررسی دقیق مراحل اجرای دستور vectorAdd می‌پردازیم. فرایند اجرای این دستور روی کارت گرافیک به سه بخش اصلی تقسیم می‌شود که در ادامه هر یک را به صورت مجزا تحلیل می‌کنیم:

۱-۲-۴ صدور فرامین توسط پردازنده مرکزی (CUDA API)

در اولین مرحله از تحلیل عملکرد برنامه، به بررسی زمان‌بندی و رفتار پردازنده مرکزی (CPU) در مدیریت کارت گرافیک می‌پردازیم. پیش از آنکه پردازنده گرافیکی بتواند محاسبات ریاضی خود را آغاز کند، پردازنده مرکزی موظف است فضای مورد نیاز را در حافظه کارت گرافیک رزرو کرده، داده‌های ورودی را به آن منتقل کند و در نهایت فرمان آغاز پردازش را صادر نماید. این مجموعه عملیات کنترلی از طریق رابط برنامه‌نویسی کودا (CUDA API) انجام می‌گیرد.

جدول ۱ خلاصه عملکرد فراخوانی‌های CUDA API را بر اساس زمان صرف‌شده در سمت

پردازنده مرکزی نشان می‌دهد. این داده‌ها به کمک ابزار Nsight Systems استخراج شده‌اند و دیدگاه روشنی از سربار عملیات کنترلی پیش از اجرای محاسبات به دست می‌دهند.

جدول ۱: خلاصه زمان‌بندی فراخوانی‌های CUDA API در سمت پردازنده مرکزی

Name	Time (%)	Total Time (ns)	Num Calls	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)
cudaMalloc	97.9	114197892	3	38065964.0	4705.0	1324	114191863	65926962.4
cuLibraryLoadData	1.3	1476459	1	1476459.0	1476459.0	1476459	1476459	0.0
cudaMemcpy	0.3	404951	3	134983.7	37773.0	37692	329486	168444.0
cuLibraryGetKernel	0.3	308947	1	308947.0	308947.0	308947	308947	0.0
cudaLaunchKernel	0.1	163342	1	163342.0	163342.0	163342	163342	0.0
cudaFree	0.1	62111	3	20703.7	7262.0	1894	52955	28059.1
cuModuleGetLoadingMode	0.0	1747	1	1747.0	1747.0	1747	1747	0.0
cuKernelGetName	0.0	243	1	243.0	243.0	243	243	0.0

با تحلیل داده‌های جدول ۱، نتایج زیر استنتاج می‌شود:

- **گلوگاه تخصیص حافظه:** دستور cudaMalloc به تنهایی 97.9 درصد از کل زمان درگیری پردازنده مرکزی (معادل حدود 114 میلی‌ثانیه) را به خود اختصاص داده است. این دستور 3 بار فراخوانی شده است که نشان‌دهنده تخصیص فضا برای دو بردار ورودی و یک بردار خروجی است. این آمار نشان می‌دهد که عملیات رزرو حافظه در سیستم عامل و درایور کارت گرافیک، یک عملیات بسیار زمان‌بر و پرهزینه محسوب می‌شود.
- **سربار انتقال داده:** دستور cudaMemcpy که وظیفه جابه‌جایی اطلاعات بین حافظه اصلی (RAM) و حافظه گرافیکی (VRAM) را بر عهده دارد، با 0.3 درصد از زمان کل (حدود 404 میکروثانیه) در رتبه‌های بعدی قرار گرفته است. این دستور نیز 3 بار فراخوانی شده است.
- **سرعت صدور فرمان پردازش:** در مقابل، دستور cudaLaunchKernel که مستقیماً فرمان آغاز محاسبات را به هسته‌های گرافیکی ارسال می‌کند، تنها 0.1 درصد از زمان (حدود 163 میکروثانیه) را صرف کرده است. این امر ثابت می‌کند که تاخیر ارتباطی پردازنده مرکزی برای صدور فرمان بسیار ناچیز است.

این مشاهدات، زمینه‌ساز درک بهتر گلوگاه‌های معماری در سناریوهای پیچیده‌تر است. در کاربردهایی مانند اجرای مدل‌های زبانی بزرگ که با کمبود حافظه گرافیکی مواجه می‌شوند (حالت Offloading)، فراخوانی‌های مکرر برای تخصیص و انتقال داده‌ها به شدت افزایش می‌یابد. نتایج این گام اثبات می‌کند که سربار مدیریت حافظه می‌تواند پیش از آنکه گرافیک به مرز توان پردازشی خود برسد، به گلوگاه اصلی سیستم تبدیل گردد.

۴-۲-۲ اجرای محاسبات درون هسته‌های گرافیکی (GPU Kernel)

پس از تکمیل عملیات کنترلی و انتقال داده‌ها توسط پردازنده مرکزی، هسته‌های گرافیکی پردازش اصلی (عملیات ریاضی) را آغاز می‌کنند. جدول ۲ خلاصه عملکرد اجرای کرنل محاسباتی را مستقیماً بر روی پردازنده گرافیکی نشان می‌دهد.

جدول ۲: خلاصه زمان‌بندی اجرای کرنل‌های پردازشی روی کارت گرافیک

Name	Time (%)	Total Time (ns)	Instances	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)
vectorAdd(const float *, const float *, float *, int)	100.0	2081	1	2081.0	2081.0	2081	2081	0.0

با تحلیل داده‌های جدول ۲ و مقایسه آن با مرحله قبل، نتایج کلیدی زیر به دست می‌آید:

- **زمان اجرای محاسبات:** همان‌طور که مشاهده می‌شود، کرنل vectorAdd تنها یک بار (1 نمونه) فراخوانی شده و کل زمان اجرای آن روی هسته‌های پردازنده گرافیکی برابر با 2081 نانوثانیه (تنها حدود 2 میکروثانیه) است. این کرنل 100.0 درصد از زمان پردازش خالص گرافیکی در این برنامه ساده را به خود اختصاص داده است.
- **مقایسه سربار ارتباطی و توان پردازشی:** مقایسه این زمان با فاز صدور فرامین (جدول ۱) اختلاف بسیار چشمگیری را نمایان می‌کند. در حالی که تخصیص حافظه در سمت پردازنده مرکزی حدود 114 میلی‌ثانیه زمان برده بود، اجرای واقعی عملیات ریاضی بر روی کارت گرافیک در کسری از میلی‌ثانیه (حدود 0.002 میلی‌ثانیه) به پایان رسیده است.
- **نتیجه‌گیری در خصوص گلوگاه‌ها:** این اختلاف فاحش ثابت می‌کند که در کاربردهایی با حجم داده‌ی کم، گلوگاه سیستم به هیچ‌وجه توان محاسباتی هسته‌های GPU نیست؛ بلکه سربار ناشی از رابط برنامه‌نویسی (CUDA API)، سیستم‌عامل و مدیریت حافظه است که بیشترین زمان اجرای برنامه را به خود اختصاص می‌دهد. این مسئله در مدل‌های زبانی بزرگ اهمیت دوچندان پیدا می‌کند، زیرا جابجایی مداوم وزن‌های مدل بین حافظه اصلی و کارت گرافیک می‌تواند منجر به قفل شدن سیستم در گلوگاه ارتباطی (به جای گلوگاه پردازشی) شود.

۴-۲-۳ انتقال داده‌ها در گذرگاه ارتباطی (GPU MemOps)

مرحله نهایی در چرخه اجرای یک برنامه بر روی کارت گرافیک، جابه‌جایی فیزیکی داده‌ها میان حافظه اصلی سیستم (RAM) و حافظه کارت گرافیک (VRAM) از طریق گذرگاه PCIe است. این عملیات شامل انتقال دوطرفه داده‌ها؛ یعنی «میزبان به دستگاه» (Host-to-Device) برای ارسال

پرامپت‌ها و وزن‌های مدل، و «دستگاه به میزبان» (Device-to-Host) برای دریافت توکن‌های خروجی و نتایج است که در صورت محدودیت حافظه، به یکی از گلوگاه‌های اصلی سیستم تبدیل می‌شود. جداول ۳ و ۴ به ترتیب زمان صرف‌شده و حجم داده‌های منتقل‌شده را در عملیات حافظه نشان می‌دهند.

جدول ۳: خلاصه زمان‌بندی عملیات انتقال داده در گذرگاه PCIe

Operation	Time (%)	Total Time (ns)	Count	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)
[CUDA memcpy Host-to-Device]	67.9	33927	2	16963.5	16963.5	16835	17092	181.7
[CUDA memcpy Device-to-Host]	32.1	16067	1	16067.0	16067.0	16067	16067	0.0

جدول ۴: خلاصه حجم عملیات انتقال داده در گذرگاه PCIe

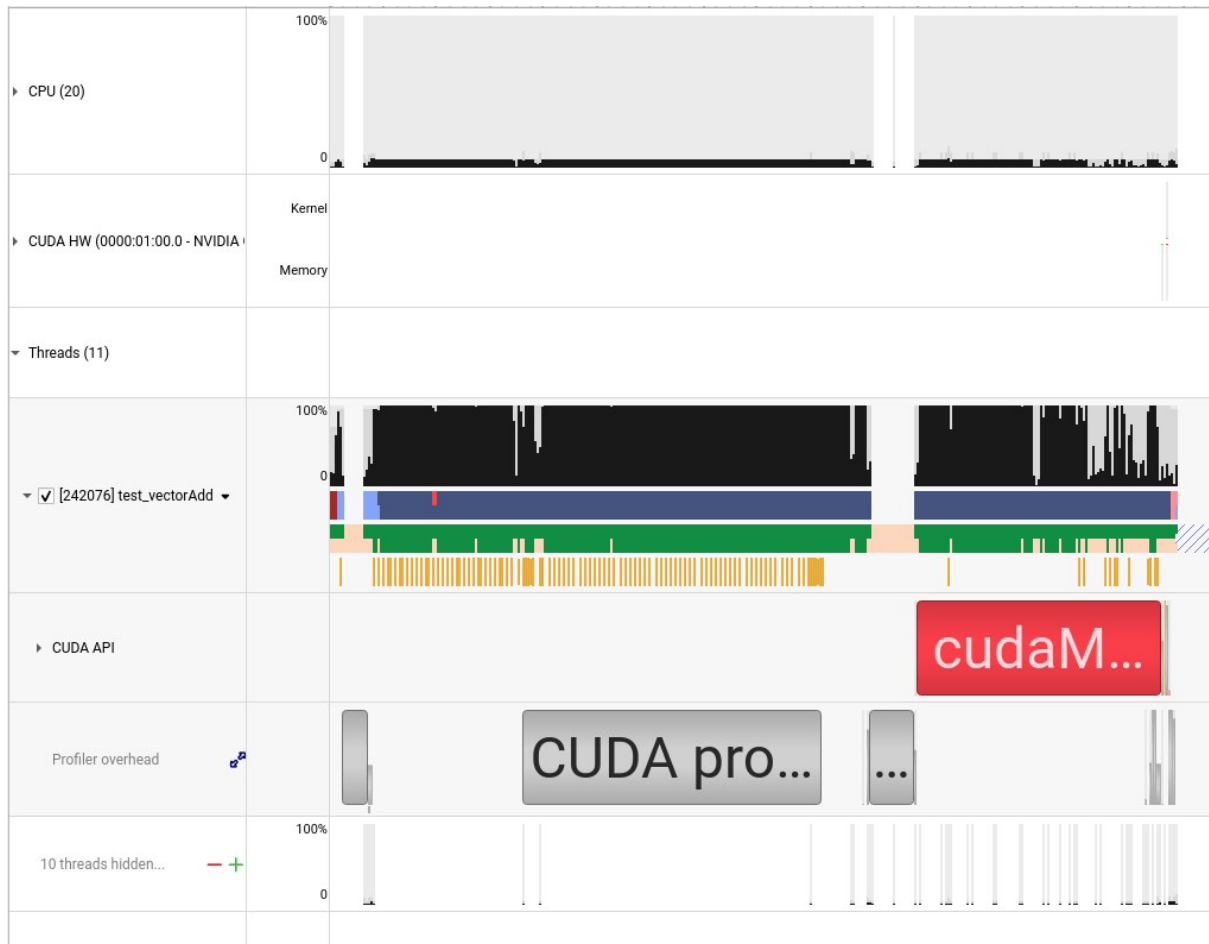
Operation	Total (MB)	Count	Avg (MB)	Med (MB)	Min (MB)	Max (MB)	StdDev (MB)
[CUDA memcpy Host-to-Device]	0.400	2	0.200	0.200	0.200	0.200	0.000
[CUDA memcpy Device-to-Host]	0.200	1	0.200	0.200	0.200	0.200	0.000

با تحلیل همزمان این دو جدول و مقایسه آن با مراحل قبل، به نتایج بسیار مهمی در زمینه معماری سیستم دست می‌یابیم:

- **تعداد و حجم انتقال:** جدول ۴ نشان می‌دهد که عملیات کپی داده‌ها به کارت گرافیک (Host-to-Device) دقیقاً ۲ بار انجام شده است که با دو بردار ورودی برنامه vectorAdd مطابقت دارد. مجموع حجم این انتقال 0.400 مگابایت (هر بردار 0.200 مگابایت) است. در نقطه مقابل، خروجی نهایی (Device-to-Host) طی 1 عملیات و با حجم 0.200 مگابایت به حافظه اصلی بازگشته است.
- **زمان صرف‌شده در گذرگاه:** بر اساس جدول ۳، انتقال بردارهای ورودی روی هم رفته حدود 33.9 میکروثانیه (معادل 33927 نانوثانیه) و بازگرداندن نتیجه حدود 16.1 میکروثانیه زمان برده است. مجموعاً انتقال فیزیکی این مقدار اندک از داده‌ها حدود 50 میکروثانیه طول کشیده است.
- **نتیجه‌گیری نهایی (گلوگاه انتقال داده):** با مقایسه این 50 میکروثانیه با زمان اجرای واقعی محاسبات در هسته‌های گرافیکی (که طبق جدول قبل تنها 2 میکروثانیه بود)، متوجه می‌شویم که زمان صرف‌شده برای جابه‌جایی فیزیکی داده‌ها 25 برابر بیشتر از زمان پردازش ریاضی آن‌ها بوده است!

این تحلیل به خوبی یکی از بزرگترین چالش‌های اجرای مدل‌های زبانی بزرگ را توجیه می‌کند. هنگامی که از قابلیت CPU Memory Offloading برای جبران کمبود حافظه گرافیکی استفاده

می‌شود، حجم عظیمی از وزن‌های مدل (در مقیاس گیگابایت) باید به طور مداوم از طریق گذرگاه PCIe منتقل شوند. همان‌طور که این گام نشان داد، سرعت پایین گذرگاه ارتباطی نسبت به هسته‌های پردازشی باعث می‌شود که کارت گرافیک زمان زیادی را در حالت بیکاری منتظر رسیدن داده‌ها بماند و گلوگاه اصلی سیستم از «توان پردازشی» به «پهنای باند ارتباطی» تغییر یابد.



شکل ۲: خروجی داخل محیط گرافیکی Nsight System

۳-۴ گام ۳: اجرای مدل، پروفایلینگ سخت‌افزاری و ارزیابی عملکرد

۱-۳-۴ مشخصات کلی مدل و تنظیمات پایه

- **model name :** Qwen/Qwen2.5-1.5B-Instruct : انتخاب مدل سبک و بهینه‌ی Qwen با 1.5 میلیارد پارامتر نسخه دستوری‌پذیر (Instruct) برای سرویس‌دهی.

- **vram-utilization : 0.7** : مقداردهی مرجع یا سطح در نظر گرفته شده برای میزان استفاده

از حافظه VRAM.

- `max-model-len : 2500`: تعیین حداکثر طول زمینه (Context Window) روی 2500 توکن جهت مدیریت بهینه‌ی حافظه در طول پردازش درخواست‌ها.

```
VLLM_USE_V2_MODEL_RUNNER=0 nsys profile \  
--trace=cuda,nvtx,osrt,cudnn,cublas \  
--sample=none \  
--cpuctxsw=none \  
--backtrace=none \  
--cuda-graph-trace=node \  
--output=qwen_profile \  
--force-overwrite=true \  
vllm serve Qwen/Qwen2.5-1.5B-Instruct \  
--gpu-memory-utilization 0.70 \  
--max-model-len 2500 \  
--port 8000
```

شکل ۳: مشخصات مدل و تنظیمات پایه

۲-۳-۴ دستور راه‌اندازی سرور و پروفایلینگ:

- `VLLM_USE_V2_MODEL_RUNNER=0`: از این متغیر لوکال برای استفاده از ورژن قدیمی `vllm` استفاده می‌کنیم زیرا ورژن‌های جدید برای قسمت `offloading` که در گام ۴ هست مشکل داشتند و نتایج آزمایش را خراب می‌کردند.
- `nsys profile`: ابزار `NVIDIA Nsight Systems` را برای ثبت دقیق عملکرد پردازنده گرافیکی اجرا می‌کند.
- `--trace=cuda,nvtx,osrt,cudnn,cublas`: به پروفایلر می‌گوید دقیقاً چه چیزهایی را زیر نظر بگیرد. در اینجا توابع پایه `CUDA`، توابع سیستم‌عامل (`osrt`) و کتابخانه‌های بهینه‌ساز یادگیری عمیق انویدیا (`cuDNN` و `cuBLAS`) ردیابی می‌شوند.
- `--sample=none \ --cpuctxsw=none \ --backtrace=none`: این 3 خط، نمونه‌برداری‌های مربوط به `CPU` (مثل ردگیری جابجایی بین پردازش‌ها یا همان `Context Switch`) را خاموش

می‌کنند. دلیل این کار کاهش سربار (Overhead) پروفایلر است تا داده‌های ثبت‌شده برای کارت گرافیک با بالاترین دقت و کمترین دخالت ثبت شوند.

- `--cuda-graph-trace=node`: از آنجا که vLLM برای افزایش سرعت از ویژگی CUDA Graphs استفاده می‌کند، این سوییچ باعث می‌شود عملکرد هر نود (Node) در داخل این گراف‌ها به طور مجزا ثبت شود.

- `--output=qwen_profile`: فایل خروجی پروفایل را با این نام ذخیره می‌کند (که معمولاً با پسوند `.nsys-rep` ساخته می‌شود).

- `--force-overwrite=true`: اگر قبلاً فایلی با نام بالا ساخته شده باشد، بدون پرسش آن را بازنویسی می‌کند.

- `vllm serve Qwen/Qwen2.5-1.5B-Instruct`: بخش اصلی دستور که سرور مدل Qwen با 1.5 میلیارد پارامتر را اجرا می‌کند.

- `--gpu-memory-utilization 0.70`: به vLLM اجازه می‌دهد دقیقاً 70 درصد از حافظه VRAM اختصاصی کارت گرافیک (در اینجا NVIDIA GeForce RTX 4060 با ظرفیت 8 گیگابایت) را به وزن‌های مدل و حافظه کش (KV Cache) اختصاص دهد.

- `--max-model-len 2500`: طول پنجره‌ی کانتکست را روی 2500 توکن قفل می‌کند تا از پر شدن ناگهانی حافظه (OOM) جلوگیری شود.

- `--port 8000`: سرور روی پورت 8000 سیستم شما گوش به زنگ درخواست‌ها می‌ماند.

```
guidellm run \
--backend kind=openai_http,target="http://localhost:8000/v1",model="Qwen/Qwen2.5-1.5B-Instruct" \
--data '{"kind":"synthetic_text","prompt_tokens":128,"output_tokens":64,"prefix_buckets":[{"bucket_weight":100,"prefix_count":2,"prefix_tokens":2000}]}' \
--profile kind=concurrent,streams=4 \
--constraint kind=max duration,seconds=60 \
--output kind=json,path="/guidellm_results.json"
```

شکل ۴: دستور فشارسنجی و تولید بار پردازشی

۳-۳-۴ دستور فشارسنجی و تولید بار پردازشی

این دستور با استفاده از ابزار GuideLLM، یک سناریوی شبیه‌سازی شده را با حجم زیادی از درخواست‌ها به سمت سرور می‌فرستد تا عملکرد آن را بسنجد.

- `guidellm run`: اجرای تست بنچمارک.

- `--backend kind=openai_http,target="http://localhost:8000/v1",model="..."`
نوع اتصال را مشخص می‌کند. به GuideLLM می‌گوید از استاندارد API اوپن‌ای‌ای استفاده کند و درخواست‌ها را به پورت 8000 (که در مرحله قبل ساختید) بفرستد و نام مدل را هم به آن پاس می‌دهد.

- `--data '{"kind":"synthetic_text",...}'`: این بخش بسیار مهم است و شکل داده‌های تستی را تعریف می‌کند:

- `prompt_tokens: 128`: طول هر سوالی که به مدل فرستاده می‌شود 128 توکن است.
- `output_tokens: 64`: از مدل انتظار می‌رود دقیقاً 64 توکن جواب تولید کند.
- `prefix_buckets... prefix_tokens: 2000`: این بخش یک پیش‌زمینه (مثل یک دستورالعمل ثابت یا System Prompt طولانی) با طول 2000 توکن را شبیه‌سازی می‌کند. این کار به شدت روی سیستم Prefix Caching در vLLM فشار می‌آورد و عملکرد آن را تست می‌کند.

- `--profile kind=concurrent,streams=4`: نوع ارسال بار را "همزمان" (Concurrent) تعیین می‌کند؛ به این معنی که ابزار GuideLLM همیشه تلاش می‌کند دقیقاً 4 درخواست به صورت همزمان در حال پردازش باشند و تا یکی تمام شد، فوراً درخواست بعدی را می‌فرستد.
- `--constraint kind=max_duration,seconds=60`: شرط پایان تست است. یعنی فارغ از اینکه چند درخواست کامل شده‌اند، تست دقیقاً بعد از 60 ثانیه به پایان برسد.

- `--output kind=json,path="./guidellm_results.json"`: تمام نتایج (مثل زمان اولین توکن، سرعت تولید توکن، تاخیر و...) را با فرمت JSON در فایل در همان مسیر ذخیره می‌کند تا بعداً بتوانیم آن را تحلیل کنیم.

۴-۳-۴ مانیتور وضعیت RAM ، GPU ، CPU

با استفاده از یک اسکریپت پایتون در هر نیم ثانیه نمونه‌گیری می‌کنیم تا مقادیر رو مانیتور کنیم. کتابخانه‌های استفاده شده برای مانیتورینگ به شرح زیرند:

- `import psutil`: کتابخانه‌ی استاندارد پایتون برای سنجش وضعیت سخت‌افزاری سیستم عامل (شامل درصد درگیری CPU و میزان مصرف RAM).

- `import pynvml`: رابط پایتونی برای ابزار NVIDIA Management Library (NVML) که مستقیم با درایور کارت گرافیک ارتباط می‌گیرد و دقیق‌ترین آمار درگیری و حافظه VRAM را استخراج می‌کند.

۵-۳-۴ معیارهای عملکردی (Performance Metrics)

- **Time to First Token (TTFT)**: میانگین زمان دریافت اولین توکن خروجی پس از ارسال درخواست، 103.09 میلی‌ثانیه (مقدار دقیق: 103.0948 میلی‌ثانیه) محاسبه شده است که نشان‌دهنده سرعت بالای فاز Prefill در مدل است.
- **Server Output Throughput**: میانگین نرخ تولید توکن‌های خروجی توسط سرور vLLM برای تمام درخواست‌های همزمان، 251.47 توکن در ثانیه (مقدار دقیق: 251.4684 توکن بر ثانیه) است.
- **Server Input Throughput**: سرعت پردازش توکن‌های ورودی در فاز Prefill معادل 8,544.90 توکن در ثانیه ثبت شده است. این نرخ بسیار بالا (بیش از ۸۰۰۰ توکن بر ثانیه) به هیچ وجه نشان‌دهنده توان محاسباتی خالص هسته‌های پردازشی برای محاسبه تمام توکن‌ها از صفر نیست؛ بلکه مستقیماً ناشی از فعال بودن مکانیزم Prefix Caching و وجود توکن‌های ثابت پیش‌زمینه (مانند پرامپت‌های تکرارپذیر با میانگین 2,141 توکن) در بنچمارک است که کش KV آن‌ها به سرعت از حافظه بازخوانی شده است.
- **Total Token Throughput**: مجموع نرخ پردازش توکن‌های ورودی و خروجی در ثانیه، 8,666.10 توکن در ثانیه ثبت شده است.
- **Requests Per Second (RPS)**: تعداد درخواست‌های کامل‌شده در هر ثانیه معادل 3.86 درخواست است.

۶-۳-۴ معیارهای مانیتورینگ سخت‌افزار (Hardware Monitoring)

- **GPU Utilization (درگیری کارت گرافیک)**: کارت گرافیک در کل 60 ثانیه به مدت حدود 17.1 ثانیه بین 91% تا 100% نوسان دارد (میانگین 99.27% — که عملاً اشباع کامل حساب می‌شود، اما نه دقیقاً ثابت 100%).

- **GPU Memory Usage (مصرف حافظه گرافیکی):** حافظه کارت گرافیک در حدود ۶ گیگابایت اشغال شده است. با توجه به اینکه در دستور، سهم حافظه روی 70 درصد تنظیم شده بود (که معادل 5.6 گیگابایت از 8 گیگابایت VRAM کارت RTX 4060 است)، ۴۰۰ مگابایت باقیمانده به طور پیش فرض توسط درایور و سیستم عامل در زمان راه اندازی سیستم اشغال شده است؛ بنابراین، مدیریت حافظه دقیقاً مطابق درخواست انجام شده است.

- **CPU Utilization (درگیری پردازنده مرکزی):** بار پردازشی CPU بسیار پایین بوده و میانگین آن 5.14% (با حداکثر 10.6%) ثبت شده است؛ این موضوع نشان می دهد کل بار محاسباتی روی GPU قرار داشته و CPU صرفاً به انجام وظایف مدیریتی پرداخته است.

- **Main Memory Usage (مصرف حافظه اصلی):** مصرف حافظه اصلی سیستم عامل در طول بنچمارک با یک روند صعودی ملایم از حدود 10.5 گیگابایت آغاز شده و تا نزدیک 12.2 گیگابایت افزایش یافته است که با پایان بار پردازشی، مجدداً به حالت پایدار بازگشته است.

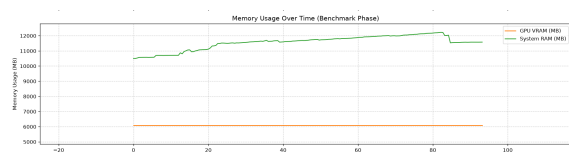
- **CUDA Kernel Execution Time (زمان اجرای کرنل های CUDA):**

– زمان کل اجرای کرنل ها: 60 ثانیه

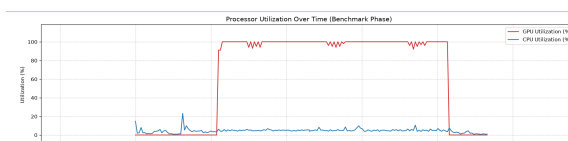
– تعداد کل فراخوانی ها: 1,507,016 بار

– میانگین زمان اجرای هر کرنل: 39.9 میکروثانیه

سنگین ترین کرنل، یک کرنل GEMM از نوع cutlass WMMA tensor-op bf16 است که 53.9% از کل زمان اجرا (32.45 ثانیه) را به خود اختصاص داده است.



شکل ۶: مانیتورینگ سخت افزار - بخش دوم



شکل ۵: مانیتورینگ سخت افزار - بخش اول

۴-۴ گام 4: پروفایلینگ و ارزیابی عملکرد مدل هنگام offload بخشی از آن به Main Memory

راه اندازی vllm و اعمال بار با guidellm

دستورات کاملاً مشابه گام ۳ هستند با این تفاوت که فلگ -cpu-offload-gb را به vllm اضافه کردیم که ۲ گیگابایت از مدل را به رم منتقل کند. برای اندازه گیری PCIe Bandwidth و میزان انتقال داده بین cpu ، gpu هم کد اسکرپت قبلی را برای این مرحله اپدیت کردیم که مقادیر جدید را هم داخل فایل csv خروجی دهد.

4.4.6. معیارهای عملکردی (Performance Metrics)

جدول ۵: Performance Metrics Summary

Metric	Value
Successful Requests	17 out of 20
Average TTFT	2560.58 ms
Average Server Output Throughput	20.10 token/s
Average Server Input Throughput	816.70 token/s
Total Token Throughput	695.82 token/s
RPS	0.27 request/sec
Average Prompt Length	2141 tokens
Request Latency (Median / p95)	13.01 / 18.37 sec
ITL (Median / p95)	180.56 / 199.70 ms
TPOT (Median / p95)	203.22 / 287.03 ms

4.4.7. معیارهای مانیتورینگ سخت افزار

- **GPU Utilization**: میانگین 98.59% (بین 0% تا 100%); تنها یک نمونه‌ی صفر در همان لحظه‌ی شروع بارگذاری ثبت شده که نویز راه‌اندازی است). این عدد بالا به معنای بهره‌وری بالا یا انجام کار مفید توسط هسته‌های پردازشی (SM) نیست؛ بلکه نشان می‌دهد کنترلر حافظه و واحدهای ارتباطی کارت گرافیک به طور مداوم درگیر مدیریت ترافیک گذرگاه

PCIe و انتظار برای انتقال وزن‌ها از حافظه اصلی بوده‌اند. در واقع، کندی سیستم ناشی از بیکار بودن (Idle) سخت‌افزار نیست، بلکه ناشی از پدیده Memory Stall و معطلی پردازنده گرافیکی برای رسیدن داده‌ها است.

- **GPU Memory**: ثابت روی 6042.56 مگابایت (در مقابل 6080.56 در گام 3 — فقط حدود 38 مگابایت کمتر؛ یعنی offload کردن 2 گیگابایت وزن، مصرف VRAM گزارش شده توسط NVML را چندان کاهش نداده، چون بخش بزرگ حافظه‌ی رزرو شده مربوط به pool ثابت KV Cache با 0.70 gpu-memory-utilization — است).

- **CPU Utilization**: میانگین 3.61٪، حداکثر 10.5٪.

- **RAM سیستم**: میانگین 14,625.78 مگابایت (بین 14,462.7 تا 14,763.3 مگابایت) — به‌طور قابل توجهی بالاتر از گام 3 (که بین 10.5 تا 12.2 گیگابایت بود)؛ این افزایش پایه‌ای حدود 2 تا 4 گیگابایتی، مستقیماً ناشی از نگهداری بخش offload شده‌ی مدل روی RAM سیستم است.

- **PCIe TX (GPU → CPU)**: میانگین 1,665.62 MB/s، حداکثر 4,755.12 MB/s.

- **PCIe RX (CPU → GPU)**: میانگین 13,760.11 MB/s، حداکثر 16,193.58 MB/s.

4.4.8. تحلیل کرنل‌های CUDA

- تعداد انواع کرنل: 91

- زمان کل اجرای کرنل‌ها: 79.12 ثانیه (در مقابل 60.16 ثانیه در گام 3 — به‌رغم کوتاه‌تر بودن تعداد درخواست‌های پردازش شده!)

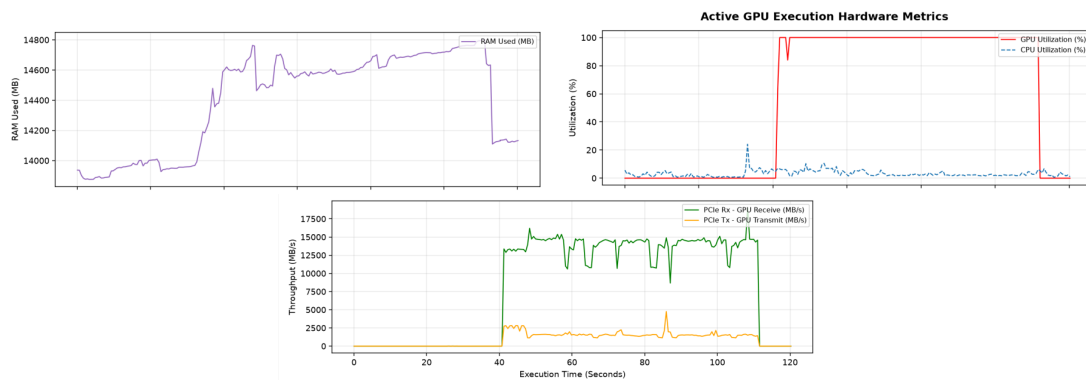
- تعداد کل فراخوانی‌ها: 111,009 (در مقابل 1,507,016 در گام 3 — طبیعی است چون تعداد درخواست موفق این بار خیلی کمتر است)

- میانگین زمان اجرای هر کرنل: 712.69 میکروثانیه (در مقابل 39.9 میکروثانیه در گام 3 — حدود 18 برابر کندتر؛ این عدد شاهد کمی مستقیم برای اثر offload روی هر عملیات GPU است)

• سنگین ترین کرنل منفرد:

ampere_bf16_s1688gemm_bf16_128x128_ldg8_f2f_stages_32x1_tn با 27.22 ثانیه معادل 34.4% از کل زمان.

• اگر بر اساس خانواده‌ی کرنل جمع بزنیم: مجموع کرنل‌های ampere_bf16 = 44.99 ثانیه (56.9%)، مجموع کرنل‌های cutlass wmma tensorop bf16 = 30.86 ثانیه (39.0%) — یعنی برخلاف گام 3 (که cutlass غالب بود)، این بار خانواده‌ی ampere_bf16 غالب است.



شکل ۷: تصاویر مربوط به تحلیل کرنل‌های CUDA

4.4.9 مقایسه عملکرد اجرای استاندارد (گام 3) و اجرای با Offload به پردازنده (گام 4)

جدول ۶: Comparison Between Step 3 and Step 4

Metric	Step 3	Step 4	Change Analysis
Successful Requests	100%	85%	System stability drop
Average TTFT	103.09 ms	2560.58 ms	~ 25x slower
Output Throughput	251.47 token/sec	20.10 token/sec	12.5x drop
System RAM Usage holding weights	10.5 to 12.2 GB	12.4 to 14.7 GB	Logical increase for
Average Kernel Time	39.9 μ s	712.69 μ s	18x slower

4.4.9.1 معیارهای عملکردی (Performance Metrics)

استفاده از قابلیت Offloading باعث افت شدید در تمامی معیارهای کاربرمحور شده است. زمان اولین توکن (TTFT) از حدود 100 میلی ثانیه به بیش از 2.5 ثانیه رسیده است. این تأخیر اولیه به دلیل زمان مورد نیاز برای فراخوانی اولیه وزنهای منتقل شده از RAM به GPU از طریق درگاه PCIe است. همچنین افت شدید Throughput خروجی (از 251.47 به 20.10 توکن بر ثانیه) نشان می دهد که تولید هر توکن جدید نیز با تأخیر همراه است، که در نهایت منجر به ناموفق بودن 3 درخواست (تایم‌اوت شدن) گردیده است.

4.4.9.2 مانیتورینگ سخت افزار و تخصیص حافظه

رفتار RAM و VRAM: برخلاف انتظار اولیه، مصرف VRAM در گام 4 کاهش چشمگیری نداشته است (6042 مگابایت در برابر 6080 مگابایت در گام 3). دلیل این امر، تخصیص از پیش تعیین شده و ثابت استخر حافظه برای KV Cache توسط پارامتر `gpu-memory-utilization 0.70` در vLLM است. در مقابل، افزایش مصرف RAM سیستم (رسیدن میانگین به 14.6 گیگابایت) به طور مستقیم نمایانگر بارگذاری بخش Offload شدهی مدل در حافظه اصلی سیستم است.

درگیری GPU و گلوگاه PCIe: با وجود افت شدید سرعت تولید توکن، GPU Utilization همچنان میانگین 98.59% را ثبت کرده است. این داده اثبات می کند که کندی سیستم ناشی از بیکار بودن (Idle) پردازنده گرافیکی نیست. در واقع، حجم عظیم داده های ورودی به GPU از طریق گذرگاه مادربرد (میانگین PCIe RX برابر با 13,760 MB/s) نشان می دهد که GPU به طور پیوسته در حال انتظار برای دریافت وزن ها از RAM سیستم بوده و چرخه های پردازشی آن صرف مدیریت انتقال داده ها (Data Movement) شده است، نه محاسبات خالص.

4.4.9.3 تحلیل کرنل های CUDA در سطح معماری

افزایش زمان اجرای کرنل ها: میانگین زمان اجرای هر کرنل از 39.9 میکروثانیه به 712.69 میکروثانیه (حدود 18 برابر) افزایش یافته است. این عدد، شاهد کمی و مستقیم برای اثر مخرب Offload بر عملکرد در سطح سخت افزار است. به دلیل کاهش نرخ پردازش، تعداد کل فراخوانی های کرنل نیز به شدت افت کرده است (از 1.5 میلیون به حدود 111 هزار فراخوانی).

تغییر خانواده کرنل های غالب: در اجرای استاندارد، کرنل های خانواده `cutlass wmma` (مبتنی

بر عملیات ماتریسی بهینه‌شده) غالب بودند. اما در حالت Offload، کرنل‌های ampere_bf16 (به‌ویژه توابعی که دارای پسوند ldg یا Load Global هستند) بیشترین سهم زمانی (حدود 57٪ کل زمان) را به خود اختصاص داده‌اند. سنگین‌ترین کرنل با اختصاص 34.4٪ از کل زمان اجرا، نشان می‌دهد که به دلیل تغییر محل ذخیره‌سازی داده‌ها، الگوی دسترسی به حافظه تغییر کرده و سیستم مجبور به استفاده از کرنل‌هایی با استراتژی‌های متفاوت و کندتر برای واکشی داده‌های خارج از VRAM شده است.

4.4.9.4. میزان انتقال داده بین CPU و GPU

در گام ۳ (اجرای استاندارد)، به دلیل قرارگیری کامل مدل در حافظه VRAM، هیچ‌گونه تبادل داده‌ای روی گذرگاه PCIe صورت نگرفته است (MB ~). اما در گام ۴، با فعال‌سازی مکانیزم Offload، با انتگرال‌گیری از نرخ لحظه‌ای ثبت‌شده در طول ۶۰ ثانیه‌ی بنچمارک، مجموعاً حدود 918,986.24 مگابایت (معادل 897.45 گیگابایت) داده — شامل 99,409.69 مگابایت ارسالی از GPU به میزبان یا همان TX و 819,576.54 مگابایت دریافتی از میزبان یا همان RX — بین حافظه اصلی (RAM) و کارت گرافیک جابه‌جا شده است.

4.4.9.5. پهنای باند مورد استفاده در گذرگاه PCIe

در حالی که در گام ۳ پهنای باند گذرگاه PCIe درگیر نبود، در گام ۴ با فعال‌سازی Offloading، میانگین ترافیک دریافتی پردازنده گرافیکی (PCIe RX) به 13,760.11 مگابایت بر ثانیه و پیک لحظه‌ای آن به 16,193.58 مگابایت بر ثانیه رسیده است (پیک ترکیبی هم‌زمان TX و RX نیز به 19,637.63 مگابایت بر ثانیه رسیده) که نشان‌دهنده اشباع کامل مسیر ارتباطی بین GPU و RAM است.

4.4.9.6. تغییرات زمان اجرای کرنل‌های CUDA

زمان کل اجرای کرنل‌ها از 60.16 ثانیه در گام ۳ به 79.12 ثانیه در گام ۴ افزایش یافته است. همچنین میانگین زمان اجرای هر کرنل CUDA با رشدی 1686.19 درصدی از 39.9 میکروثانیه در گام ۳ به 712.69 میکروثانیه در گام ۴ (حدود ۱۸ برابر کندتر) رسیده است که گواه تغییر الگوی دسترسی به حافظه و واکشی اطلاعات از خارج VRAM است.

4.4.9.7. تحلیل زمان‌بندی و معطلی کارت گرافیکی (GPU Wait / Stall Breakdown) (Time)

برای استخراج دقیق زمان‌های معطلی، بررسی سربار میزبان و تحلیل سطح پایین رفتار کارت گرافیک، از ابزار تحلیل پروفایلینگ انویدا استفاده شده است. دستور زیر برای استخراج گزارش تریس کرنل‌ها از فایل خروجی خام به کار رفته است:

```
nsys stats --report cuda_gpu_trace --format csv --output gpu_trace
qwen_profile.nsys-rep
```

این دستور، فایل باینری و حجیم پروفایلینگ (qwen_profile.nsys-rep) را پردازش کرده و اطلاعات زمان‌بندی دقیق تمامی رویدادها، کرنل‌های CUDA و عملیات انتقال حافظه را به صورت یک فایل جدول‌بندی‌شده با فرمت CSV استخراج می‌کند تا با اسکریپت‌های تحلیلی، وقفه‌های زمانی و معطلی‌های سخت‌افزاری محاسبه شوند.

با توجه به نتایج به‌دست‌آمده از تحلیل تریس در دو سناریو، مقایسه به شرح زیر است:

- گام ۳ (اجرای استاندارد بدون Offload): کل زمان تریس معادل 118.06 ثانیه بوده که تفکیک آن به صورت زیر است:

- محاسبات خالص (Pure Computation): معادل 53.53 ثانیه (45.34% کل زمان).
- انتظار و انتقال داده (Data Wait/Transfer): معادل 11.27 ثانیه (9.54% کل زمان).
- سربار پردازنده میزبان (Host/CPU Overhead): معادل 53.26 ثانیه (45.11% کل زمان).

- گام ۴ (با فعال‌سازی مکانیزم Offload): کل زمان تریس به دلیل جابه‌جایی مداوم وزن‌ها و داده‌ها به 175.89 ثانیه افزایش یافته که تفکیک آن به صورت زیر است:

- محاسبات خالص (Pure Computation): معادل 34.09 ثانیه (19.38% کل زمان) که نشان‌دهنده افت توان پردازشی مفید است.
- انتظار و انتقال داده (Data Wait/Transfer): معادل 46.71 ثانیه (26.56% کل زمان) که بیانگر افزایش شدید معطلی در اثر محدودیت پهنای باند PCIe است.
- سربار پردازنده میزبان (Host/CPU Overhead): معادل 95.08 ثانیه (54.06% کل زمان) به دلیل مدیریت سنگین انتقال وزن‌ها میان رم و کارت گرافیک.

مقایسه این آمار به روشن نشان می‌دهد که فعال‌سازی مکانیزم Offload زمان انتظار و انتقال داده را از 11.27 ثانیه به 46.71 ثانیه افزایش داده و سهم محاسبات مفید کارت گرافیک را به شدت کاهش داده است.

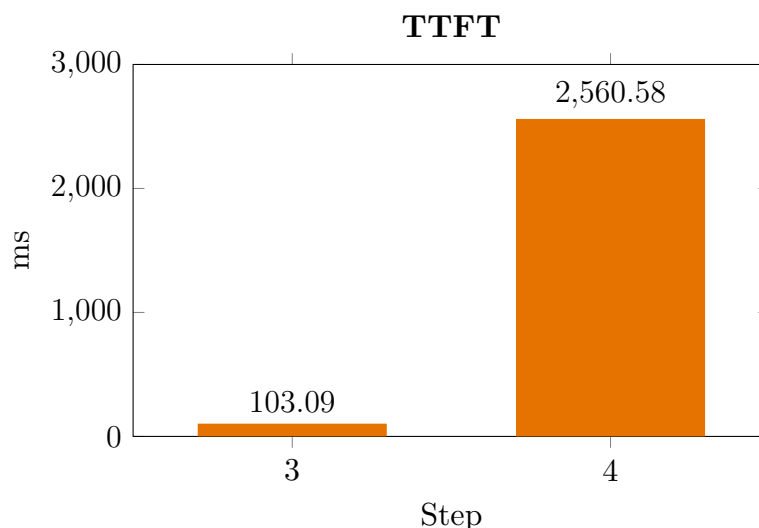
۵-۴ گام ۵: تحلیل جامع گلوگاه‌ها و ارزیابی موازنه سیستم & Synthesis) Bottle neck Analysis)

در این گام، با تجميع و مقایسه داده‌های به‌دست‌آمده از تحلیل خرد Nsight Systems (گام ۲)، ارزیابی پایه در شرایط استاندارد (گام ۳) و سنجش اثرات Offloading (گام ۴)، به تحلیل ریشه‌ای رفتار سیستم، شناسایی گلوگاه‌های سخت‌افزاری و بررسی موازنه کارایی (Trade-off) می‌پردازیم.

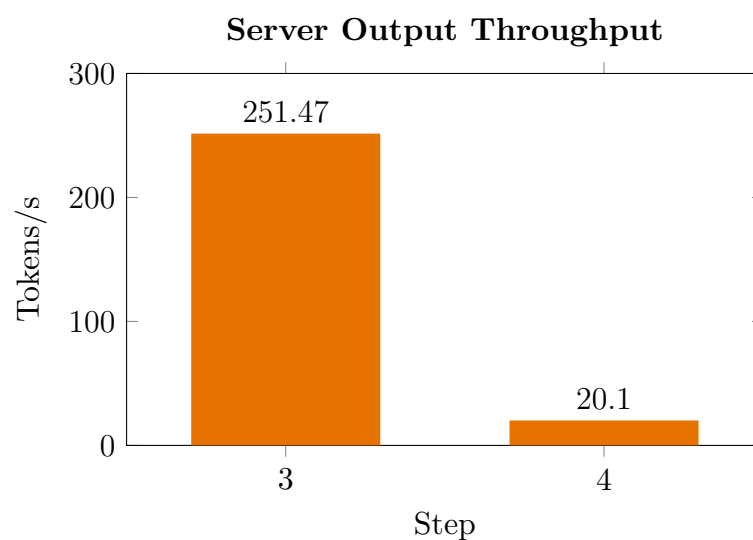
۱.۵.۴ مقایسه متریک‌های عملکردی (TTFT, Throughput, RPS)

جدول ۷: مقایسه متریک‌های عملکردی سرویس در گام‌های ۳ و ۴

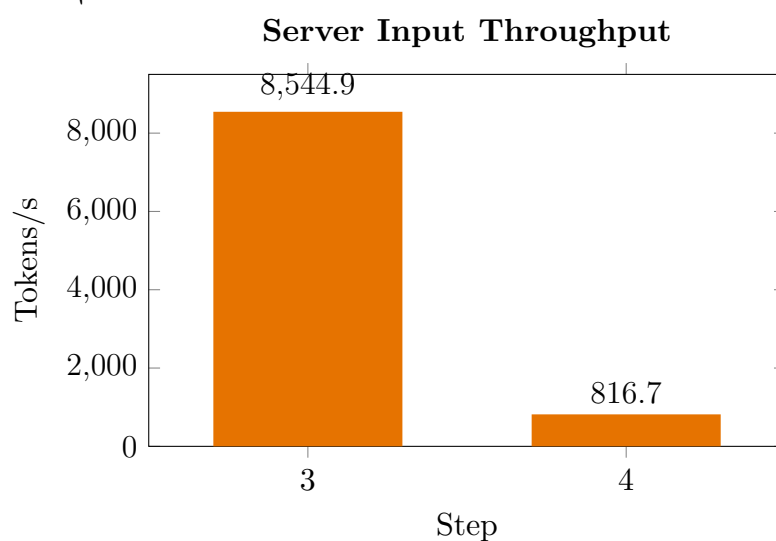
Metric	Step 3 (Standard)	Step 4 (CPU Offload)	Change (%)	Related Bottleneck
TTFT	103.09 ms	2560.58 ms	+2383.83%	PCIe Bandwidth / Memory-bound
Server Output Throughput	251.47 token/s	20.10 token/s	-92.01%	PCIe Bandwidth-bound
Server Input Throughput	8544.90 token/s	816.70 token/s	-90.44%	PCIe Bandwidth-bound
RPS	3.86 req/s	0.27 req/s	-93.01%	PCIe Bandwidth-bound



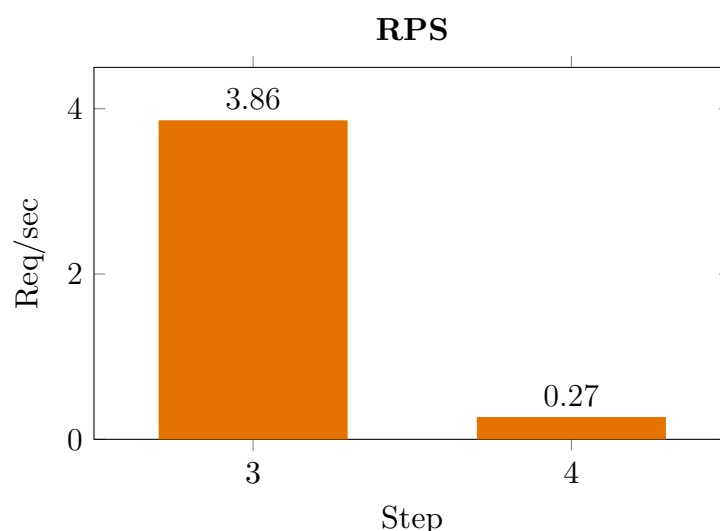
شکل ۸: نمودار مقایسه TTFT در گام‌های ۳ و ۴



شکل ۹: نمودار مقایسه Server Output Throughput در گام‌های 3 و 4



شکل ۱۰: نمودار مقایسه Server Input Throughput در گام‌های 3 و 4



شکل ۱۱: نمودار مقایسه RPS در گام‌های ۳ و ۴

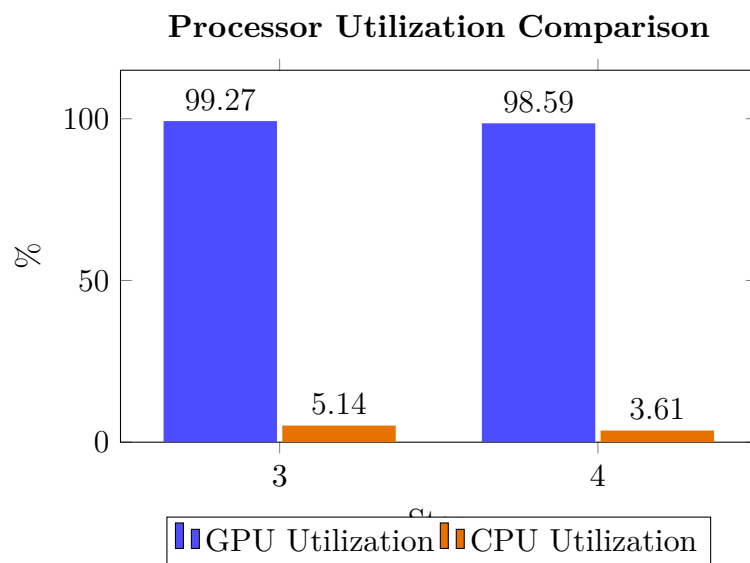
نتیجه‌گیری جدول ۷:

فعال‌سازی مکانیزم CPU Offload در گام ۴، افت بسیار شدیدی در تمامی متریک‌های سطح سرویس ایجاد کرده است. افزایش فاجعه‌بار ۲۳۸۳ درصدی در تأخیر اولین توکن (TTFT) و افت ۹۲ درصدی در توان تولید توکن (Output Throughput) نشان می‌دهد که انتقال داده‌ها بین حافظه رم سیستم و کارت گرافیک، سرعت پاسخ‌دهی سرویس‌دهنده را مختل کرده و گذرگاه ارتباطی PCIe به گلوگاه اصلی تبدیل شده است.

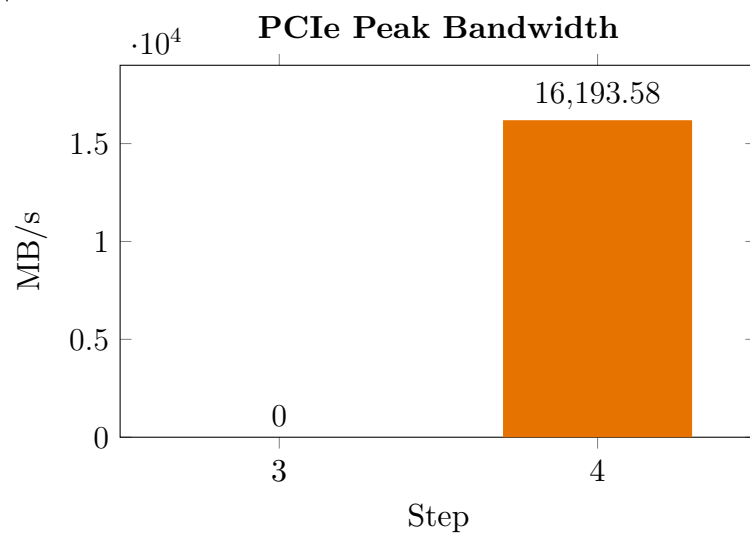
۲.۵.۴ مقایسه متریک‌های پایش سخت‌افزار و تعیین نوع گلوگاه

جدول ۸: مقایسه پارامترهای سخت‌افزاری و وضعیت گلوگاه‌ها در گام‌های ۳ و ۴

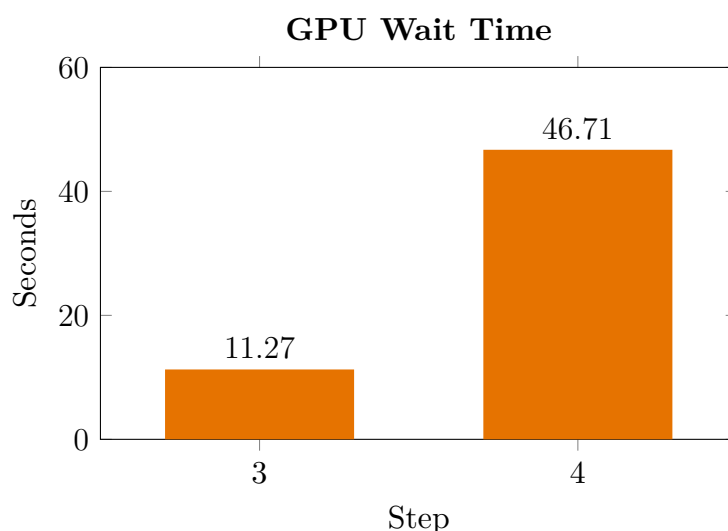
Hardware Parameter	Step 3 (Standard)	Step 4 (CPU Offload)	Bottleneck Status (Step 3)	Bottleneck Status (Step 4)
GPU Utilization	99.27%	98.59%	Compute-bound	PCIe Bandwidth-bound
VRAM Usage	6080.56 MB	6042.56 MB	GPU Memory-bound	GPU Memory-bound
CPU Utilization	5.14%	3.61%	Non-CPU-bound	Non-CPU-bound
RAM Usage	10.5 - 12.2 GB	14.6 GB	Non-Main Memory-bound	Non-Main Memory-bound
PCIe Traffic	N/A	13,760.11 MB/s	No Bottleneck	PCIe Bandwidth-bound
PCIe Peak Bandwidth	-	16,193.58 MB/s	No Bottleneck	PCIe Bandwidth-bound
GPU Wait Time	11.27 s	46.71 s	No Bottleneck	PCIe Bandwidth-bound



شکل ۱۲: نمودار مقایسه درصد درگیری پردازنده‌های GPU و CPU در گام‌های ۳ و ۴



شکل ۱۳: نمودار مقایسه پیک پهنای باند PCIe در گام‌های ۳ و ۴



شکل ۱۴: نمودار مقایسه زمان معطلی GPU در گام‌های ۳ و ۴

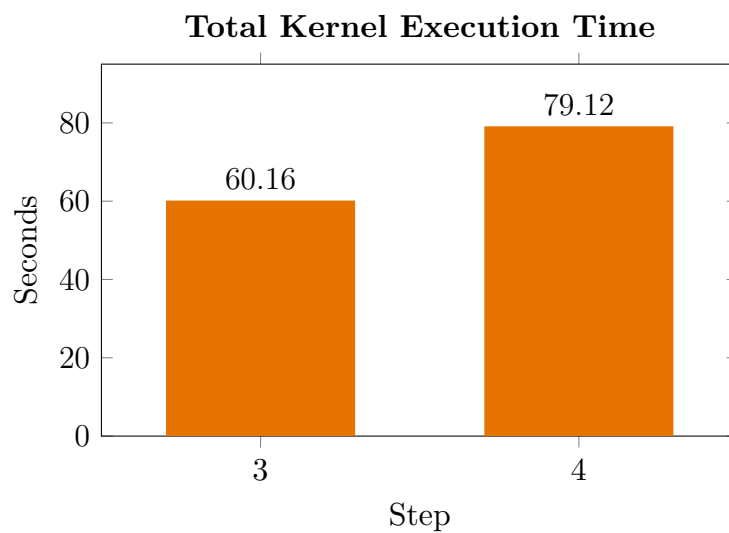
تحلیل جدول ۸:

در گام سوم به دلیل قرارگیری کامل مدل در حافظه VRAM، ترافیک سرباری روی گذرگاه PCIe اشغال نشده است. اما با فعال‌سازی Offloading در گام چهارم، میانگین ترافیک دریافتی پردازنده گرافیکی (PCIe RX) به ۷۶۰،۱۳ مگابایت بر ثانیه و پیک آن به ۱۹۳،۱۶ مگابایت بر ثانیه رسیده است. با وجود این حجم عظیم انتقال داده، پردازنده گرافیکی همچنان درگیری بالایی (۵۹.۹۸٪) دارد؛ به این معنا که GPU بیکار نیست، بلکه چرخه‌های کاری آن صرف مدیریت انتقال داده‌ها و انتظار برای واکنشی وزن‌ها از RAM شده است.

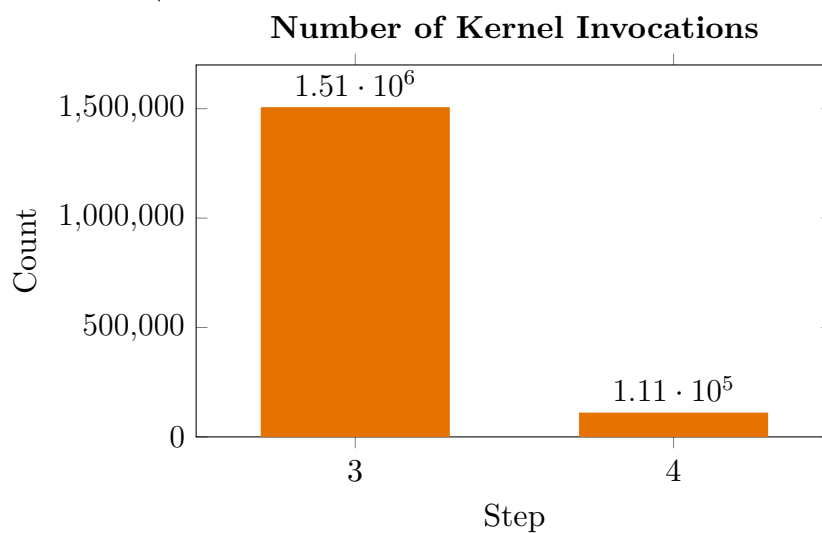
۳.۵.۴ مقایسه زمان اجرای خالص کرنل‌های CUDA (سطح پردازشی)

جدول ۹: مقایسه شاخص‌های سطح کرنل CUDA در گام‌های ۳ و ۴

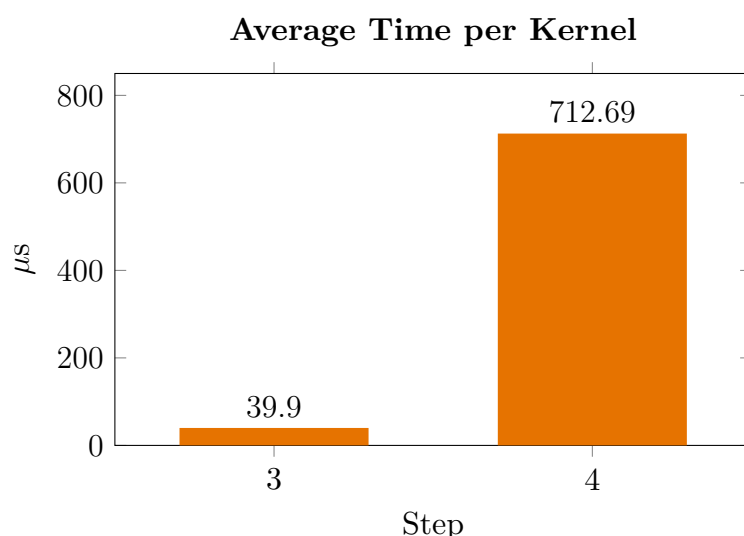
CUDA Kernel Metric	Step 3	Step 4	Change (%)	Analysis
Total Kernel Execution Time	60.16 s	79.12 s	31.52%	Total time increased due to operation slowdown
Number of Kernel Invocations	1,507,016	111,009	-92.63%	Drastic drop due to processing fewer requests
Average Time per Kernel	39.9 μ s	712.69 μ s	1686.19%	Direct impact of data fetch latency on each operation



شکل ۱۵: نمودار مقایسه زمان کل اجرای کرنل‌ها در گام‌های ۳ و ۴



شکل ۱۶: نمودار مقایسه تعداد فراخوانی کرنل‌ها در گام‌های ۳ و ۴



شکل ۱۷: نمودار مقایسه میانگین زمان اجرای هر کرنل در گام‌های ۳ و ۴

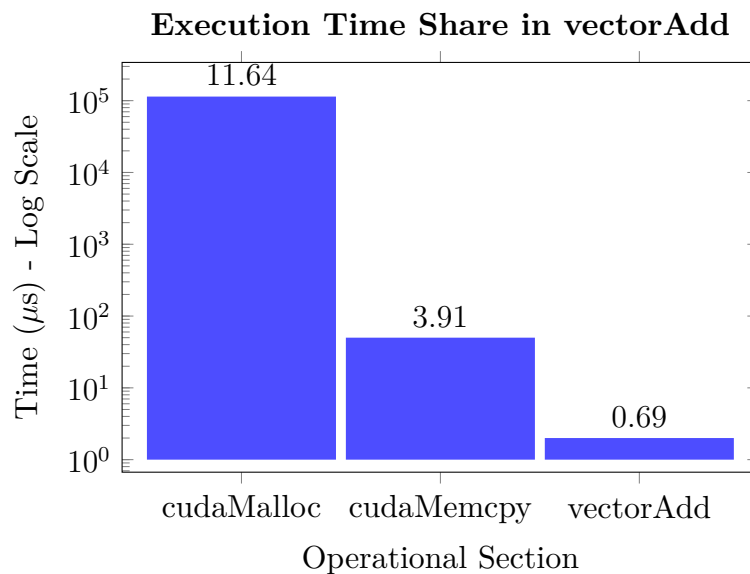
نتیجه‌گیری جدول ۹:

تحلیل سطح دقیق کرنل‌های CUDA اثبات می‌کند که میانگین زمان اجرای هر کرنل در حالت Offload بیش از 1686.19% (حدود 18 برابر) کندتر شده است. تغییر خانواده‌ی غالب کرنل‌ها از cutlass wmma (بهینه‌شده برای محاسبات ماتریسی داخل VRAM در گام ۳) به کرنل‌های ampere_bf16 با دستورات ldg (Load Global) (در گام ۴) نشان می‌دهد که الگوی دسترسی به حافظه تغییر کرده و سیستم مجبور به استفاده از استراتژی‌های کندتر برای واکشی داده‌های خارج از VRAM (حافظه اصلی سیستم) شده است.

۴.۵.۴ داده‌های خرد گام ۲ (تحلیل پایه با vectorAdd با Nsight Systems)

جدول ۱۰: تحلیل پایه با vectorAdd با Nsight Systems

Operational Section	Time / Time Share	Identified Bottleneck Type
CUDA API (cudaMalloc)	114 ms (97.9%)	OS memory allocation and control overhead
PCIe Data Transfer (cudaMemcpy)	50 μs	PCIe Bandwidth-bound (25x compute time)
GPU Math Execution (vectorAdd)	2 μs	No Compute-bound at small data volume



شکل ۱۸: نمودار مقایسه زمان اجرای بخش‌های مختلف در برنامه vectorAdd (مقیاس لگاریتمی)

نتیجه‌گیری جدول 10:

همان‌طور که در نمودار مشهود است، بیش از 97.9% از زمان کل اجرای برنامه صرف تخصیص حافظه و دستورات کنترلی درایور (cudaMalloc) شده است. با بزرگ‌نمایی بخش‌های باقی‌مانده در دایره دوم مشخص می‌شود که زمان انتقال داده روی گذرگاه PCIe (cudaMemcpy با 50 میکروثانیه) حدود 25 برابر زمان اجرای خالص محاسبات ریاضی روی هسته‌های گرافیکی (vectorAdd با 2 میکروثانیه) است. این تحلیل بصری اثبات می‌کند که در پردازش‌های گرافیکی، توان محاسباتی هسته‌ها (Compute) اصلاً گلوگاه نیست، بلکه سربار مدیریت حافظه و انتقال داده از طریق PCIe گلوگاه‌های اصلی سیستم به شمار می‌روند.

۵.۵.۴ جمع‌بندی نهایی و تحلیل موازنه سیستم (Trade-off Analysis)

با تطبیق نتایج کلان سرویس‌دهی (جدول 1) با داده‌های سخت‌افزاری (جدول 2)، سطح کرنل (جدول 3) و تحلیل خرد (جدول 4)، سیر تحول گلوگاه‌های سیستم به شرح زیر تبیین می‌شود:

- **تغییر الگوی اجرای کرنل‌ها و افت شدید سرعت محاسباتی:** داده‌های جدول 3 به‌روشنی اثبات می‌کنند که فعال‌سازی مکانیزم CPU Offload باعث تغییر خانواده‌ی غالب کرنل‌ها از cutlass wmma (عملیات سریع ماتریسی) به کرنل‌های ampere_bf16 با دستورات واکنشی حافظه (ldg) شده است. این تغییر الگوی دسترسی به حافظه، میانگین زمان اجرای هر کرنل CUDA را با رشدی 1686.19% درصدی از 39.9 میکروثانیه به 712.69 میکروثانیه (حدود 18

برابر کندتر) رسانده است.

- **تغییر حالت گلوگاه از Compute به PCIe:** در گام 3، سیستم در حالت Compute-bound قرار داشت و GPU با بهره‌وری بالا داده‌ها را از VRAM فراخوانی می‌کرد. اما در گام 4 با انتقال بخشی از وزن‌ها به RAM، حجم عظیم ترافیک دریافتی روی گذرگاه PCIe (با میانگین 13,760.11 مگابایت بر ثانیه و پیک 19,637.63 مگابایت بر ثانیه) باعث اشباع کامل گذرگاه ارتباطی شد. این پدیده منجر به ایجاد حدود 0.85 ثانیه زمان معطلی (GPU Wait/Stall) در هسته‌های پردازشی گردید و سیستم را کاملاً به حالت PCIe Bandwidth-bound منتقل کرد.
- **موازنه مهندسی (Trade-off):** اگرچه مکانیزم CPU Offload امکان بارگذاری و اجرای مدل را روی سخت‌افزار با VRAM محدود (مانند کارت گرافیک MX330) فراهم می‌سازد، اما این قابلیت به بهای افت فاجعه‌بار سطح کیفیت سرویس (QoS) تمام می‌شود. نتایج بنچمارک نشان می‌دهد این موازنه منجر به افزایش 2383.83% درصدی تاخیر پاسخ‌دهی (TTFT)، افت 92.01% درصدی توان خروجی توکن‌ها (Output Throughput) و کاهش 93.01% درصدی نرخ پردازش درخواست‌ها (RPS) شده است.

۵ قسمت امتیازی: بررسی تغییر اندازه Batch Size بر میزان استفاده از GPU، پهنای باند حافظه و عملکرد سیستم

هدف این بخش: بررسی این موضوع که اگر تعداد درخواست‌های همزمانی که کارت گرافیک در یک لحظه پردازش می‌کند (Batch Size) را کم یا زیاد کنیم، چه اتفاقی برای سرعت پاسخ‌دهی (Latency)، توان عملیاتی کل (Throughput) و میزان درگیری سخت‌افزار (GPU Utilization) می‌افتد.

۱-۰-۵ راه‌اندازی vLLM

```
serve model with : vllm serve Qwen/Qwen2.5-1.5B-Instruct --gpu-memory-utilization 0.52 --max-num-seqs 16 --max-model-len 2048 --port 8000
```

شکل ۱۹

```
vllm serve Qwen/Qwen2.5-1.5B-Instruct --gpu-memory-utilization 0.52
```

```
--max-num-seqs 16 --max-model-len 2048 --port 8000
```

توضیح پارامترها و فلگ‌ها:

- **gpu-memory-utilization 0.52**:- میزان تخصیص حافظه گرافیکی (VRAM) را روی ۵۲ درصد کل ظرفیت تنظیم می‌کند تا از خطای کمبود حافظه (OOM) جلوگیری شود.
- **max-num-seqs 16**:- حداکثر تعداد دنباله‌ها یا درخواست‌های هم‌زمان در هر بیچ (Batch) را روی ۱۶ محدود می‌کند تا مدیریت حافظه و زمان‌بندی به شکل بهینه‌تری انجام شود.
- **max-model-len 2048**:- طول زمینه (Context Length) یا مجموع توکن‌های ورودی و خروجی مدل را حداکثر روی ۲۰۴۸ توکن تنظیم می‌کند.
- **port 8000**:- پورت ارتباطی سرور را روی پورت ۸۰۰۰ تعیین می‌کند تا کلاينت‌ها و ابزارهای سنجش بتوانند به API متصل شوند.

۵-۰-۲ اجرای بنچمارک و سنجش کارایی (GuideLLM)

```
use guidllm with : guidllm run \  
--backend kind=openai_http,target="http://localhost:8000/v1",model="Qwen/Qwen2.5-1.5B-Instruct" \  
--data kind=synthetic_text,prompt_tokens=256,output_tokens=128 \  
--profile kind=constant,rate=1 \  
--constraint kind=max_requests,count=20 \  
--output kind=json,path="./guidellm_results.json"
```

شکل ۲۰: اجرای بنچمارک GuideLLM

```
guidellm run \  

```

```
--backend kind=openai_http,target="http://localhost:8000/v1",model="Qwen/Qwen2.5-1.5B-Instruct" \  
--data kind=synthetic_text,prompt_tokens=256,output_tokens=128 \  
--profile kind=constant,rate=1 \  
--constraint kind=max_requests,count=20 \  
--output kind=json,path="./guidellm_results.json"
```

دو فلگ اصلی تعیین‌کننده و تأثیرگذار بر بچ‌سایز در این دستورات عبارتند از:

- **max-num-seqs 16** در دستور راه‌اندازی سرور vLLM

- نرخ ارسال (rate=1) در پروفایل ابزار GuideLLM

چرا این دو فلگ بچسباز را تغییر می دهند؟

- **max-num-seqs-**: یک سقف سخت افزاری/نرم افزاری تعیین می کند؛ یعنی سرور اجازه ندارد در هر چرخه پردازشی، بچی بزرگتر از ۱۶ درخواست تشکیل دهد.
- **نرخ ارسال (rate=1)**: تعیین می کند کلاینت با چه سرعتی درخواست ها را می فرستد. وقتی نرخ روی ۱ درخواست بر ثانیه تنظیم می شود، تعداد درخواست های همزمان ورودی کم است، در نتیجه بچسباز واقعی در زمان اجرا کوچک می ماند.

۱-۵ کارایی (Performance)

۱-۱-۵ توان عملیات (Throughput)

Server Throughput Statistics (All Requests)						
Benchmark	Requests			Input Tokens	Output Tokens	Total Tokens
Strategy	Concurrency	Per Sec	Per Sec	Per Sec	Per Sec	Per Sec
	Mdn	Mean	Mean			
constant	2.0	2.1	0.9	299.9	121.6	392.3

شکل ۲۱: توان عملیات برای Batch Size = 1

Server Throughput Statistics (All Requests)						
Benchmark	Requests			Input Tokens	Output Tokens	Total Tokens
Strategy	Concurrency	Per Sec	Per Sec	Per Sec	Per Sec	Per Sec
	Mdn	Mean	Mean			
constant	39.0	38.3	5.0	1530.5	640.3	2066.0

شکل ۲۲: توان عملیات برای Batch Size = 8

Server Throughput Statistics (All Requests)						
Benchmark	Requests			Input Tokens	Output Tokens	Total Tokens
Strategy	Concurrency	Per Sec	Per Sec	Per Sec	Per Sec	Per Sec
	Mdn	Mean	Mean			
constant	101.0	101.7	6.8	2013.5	869.2	2804.5

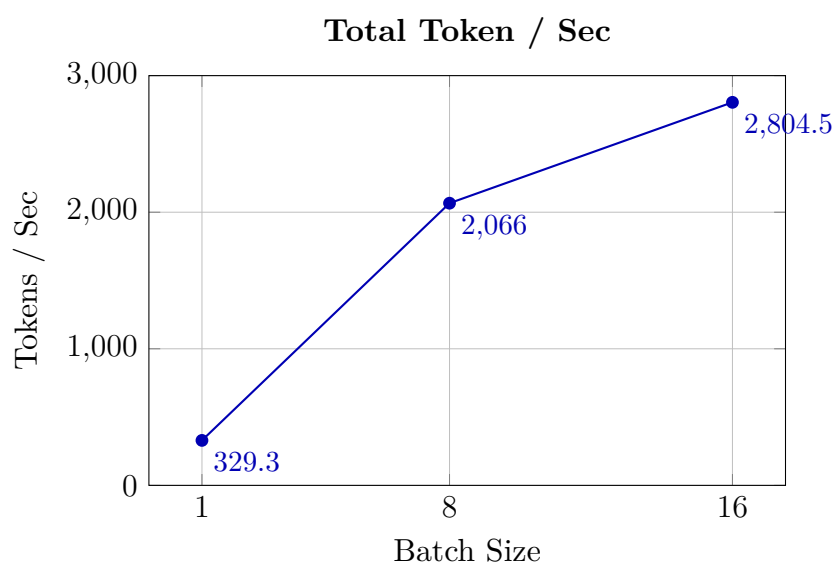
شکل ۲۳: توان عملیات برای Batch Size = 16

۲-۱-۵ تأخیر (Latency)

برای ارزیابی دقیق عملکرد سرور در مواجهه با بارهای کاری مختلف، علاوه بر میانگین تأخیر کل، بررسی معیارهای جزئی تر مانند زمان تا اولین توکن (TTFT)، زمان تولید هر توکن (TPOT) و صدک های آماری (Percentiles) ضروری است:

جدول ۱۱: جدول مربوط به Throughput

Batch Size	Avg Req/Sec	Prompt Tokens/Sec	Output Tokens/Sec	Total Tokens/Sec	Avg Concurrency
1	0.9	299.9	121.6	329.3	2.1
8	5.0	1530.5	640.3	2066.0	38.3
16	6.8	2013.5	869.2	2804.5	101.7



شکل ۲۴: نمودار تغییرات Total Tokens / Sec نسبت به اندازه دسته

```

Request Latency Statistics (Completed Requests)
=====
| Benchmark | Request Latency | TTFT | TTFOT | ITL | TPOT |
| Strategy  | Sec             | ms   | ms     | ms   | ms   |
|           | Mdn | p95 | Mdn | p95 | Mdn | p95 | Mdn | p95 | Mdn | p95 |
|-----|-----|-----|-----|-----|-----|
| constant | 2.3  | 2.3  | 104.5 | 112.2 | 104.5 | 112.2 | 17.0 | 17.1 | 17.7 | 17.8 |
=====

```

شکل ۲۵: تأخیر برای Batch Size = 1

```

Request Latency Statistics (Completed Requests)
=====
| Benchmark | Request Latency | TTFT | TTFOT | ITL | TPOT |
| Strategy  | Sec             | ms   | ms     | ms   | ms   |
|           | Mdn | p95 | Mdn | p95 | Mdn | p95 | Mdn | p95 | Mdn | p95 |
|-----|-----|-----|-----|-----|-----|
| constant | 8.0  | 12.5 | 4982.4 | 9821.1 | 4982.4 | 9821.1 | 24.0 | 24.1 | 62.7 | 97.4 |
=====

```

شکل ۲۶: تأخیر برای Batch Size = 8

```

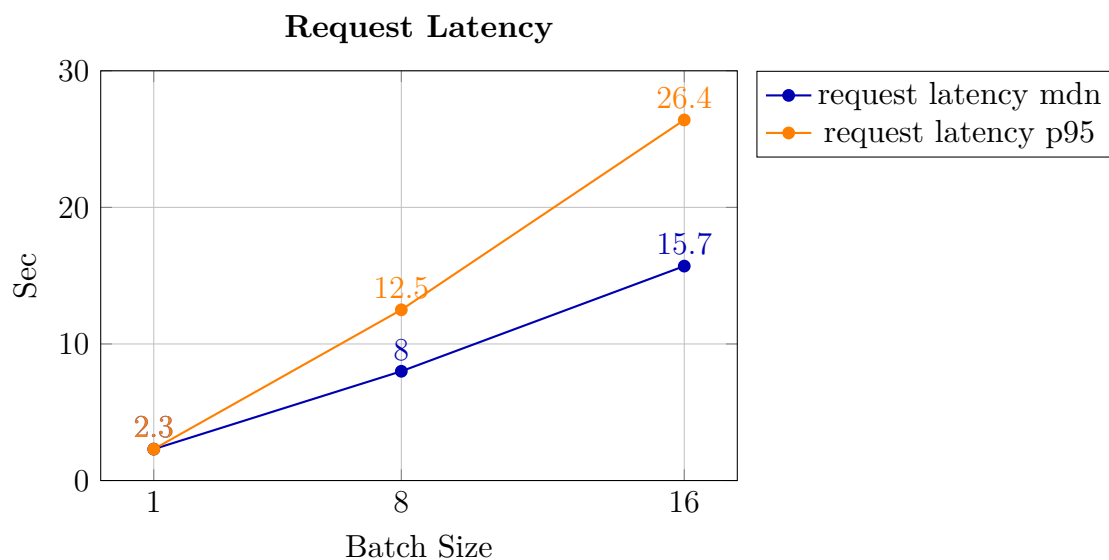
Request Latency Statistics (Completed Requests)
=====
| Benchmark | Request Latency | TTFT | TTFOT | ITL | TPOT |
| Strategy  | Sec             | ms   | ms     | ms   | ms   |
|           | Mdn | p95 | Mdn | p95 | Mdn | p95 | Mdn | p95 | Mdn | p95 |
|-----|-----|-----|-----|-----|-----|
| constant | 15.7 | 26.4 | 13389.2 | 24112.8 | 13389.2 | 24112.8 | 17.9 | 17.9 | 122.4 | 206.2 |
=====

```

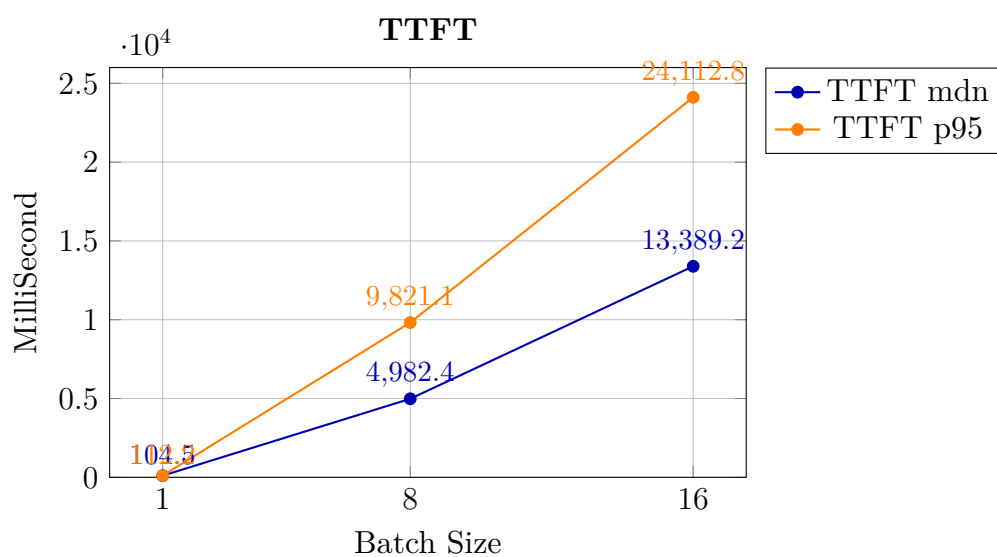
شکل ۲۷: تأخیر برای Batch Size = 16

جدول ۱۲: جدول مربوط به Latency

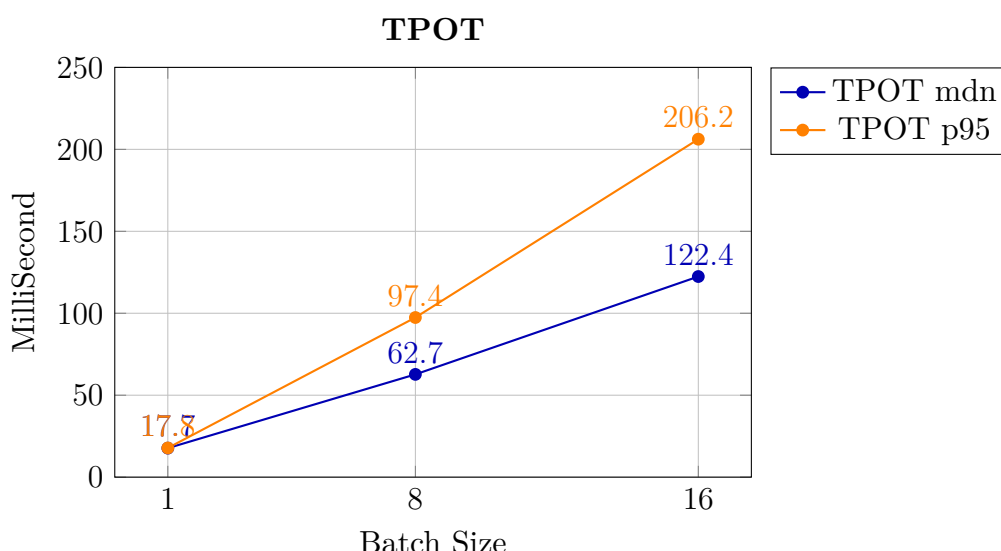
Batch Size	Total Latency (mdn)	Total Latency (p95)	TTFT (mdn)	TTFT (p95)	ITL (mdn)	TPOT (mdn)	TPOT (p95)
1	2.3	2.3	104.5	112.2	17.0	17.7	17.8
8	8.0	12.5	4982.4	9821.1	24.0	62.7	97.4
16	15.7	26.4	13389.2	24112.8	17.9	122.4	206.2



شکل ۲۸: نمودار تغییرات Request Latency (میان و صدک ۹۵) نسبت به اندازه دسته



شکل ۲۹: نمودار تغییرات TTFT (میان و صدک ۹۵) نسبت به اندازه دسته



شکل ۳۰: نمودار تغییرات TPOT (میان و صدک ۹۵) نسبت به اندازه دسته

- **تحلیل زمان تا اولین توکن (TTFT - Time to First Token):** این شاخص نشان‌دهنده مدت زمان پردازش پرامپت ورودی در فاز پیش‌محاسبه (Prefill) است. نتایج نشان می‌دهد که در $\text{Batch Size} = 1$ میان TTFT بسیار پایین و حدود 104.5 میلی‌ثانیه است؛ اما با افزایش اندازه دسته به 8 و 16، این مقدار به ترتیب به 4982.4 میلی‌ثانیه (حدود 4.98 ثانیه) و 13389.2 میلی‌ثانیه (حدود 13.39 ثانیه) افزایش می‌یابد. دلیل این رویداد تشکیل صف انتظار و رقابت درخواست‌ها برای ورود به بلوک‌های پردازشی فاز Prefill در کارت گرافیک است.
- **تحلیل زمان تولید توکن‌های خروجی (TPOT - Time Per Output Token):** این شاخص سرعت تولید کلمات متوالی در فاز Decoding را نشان می‌دهد. با افزایش تعداد درخواست‌های همزمان از 1 به 16، میان TPOT از 17.7 میلی‌ثانیه به 122.4 میلی‌ثانیه رسیده است. این افزایش ناشی از تحمیل بار محاسباتی سنگین‌تر به هسته‌های پردازشی GPU به دلیل مدیریت همزمان توکن‌های متعدد در هر گام است.
- **نقش و تحلیل صدک‌ها (Percentiles):** برای سنجش پایداری سیستم، بررسی صدک 95 اهمیت بالایی دارد. برای مثال در $\text{Batch Size} = 16$ ، در حالی که میان تأخیر کل 15.7 ثانیه است، مقدار صدک 95 تأخیر کل به 26.4 ثانیه و صدک 95 برای TTFT به حدود 24.11 ثانیه (24112.8 میلی‌ثانیه) می‌رسد. این اختلاف نشان می‌دهد که 5 درصد از درخواست‌ها با تراکم صف شدیدتری مواجه شده‌اند.

۳-۱-۵ نتایج به دست آمده

افزایش توان عملیاتی (Throughput Growth): با افزایش مقدار max-num-seqs از ۱ به ۱۶، کل توکن‌های پردازش شده در ثانیه از 329.3 به 2804.5 توکن بر ثانیه افزایش یافته است. این جهش چشمگیر نشان می‌دهد که در batch size های پایین، واحدهای پردازشی کارت گرافیک به دلیل کمبود داده‌های همزمان دچار گرسنگی منابع شده و پتانسیل موازی‌سازی سخت‌افزار به کار گرفته نمی‌شود. با افزایش batch size، درخواست‌ها در صف تجمع شده و بهره‌وری (Occupancy) هسته‌های پردازشی به حداکثر می‌رسد.

افزایش همزمانی و مدیریت صف (Concurrency): میانگین همزمانی درخواست‌ها از 2.1 در batch size = 1 به 101.7 در batch size = 16 رسیده است. این موضوع بیانگر این است که سرور توانسته است با مدیریت بهینه زمان‌بندی، از ظرفیت کامل خط‌لوله اجرایی خود استفاده کند.

تأثیر بر شاخص‌های تأخیر (Latency Impact): دستیابی به توان عملیاتی بالا بدون هزینه نبوده است. با افزایش اندازه دسته:

- شاخص TTFT (زمان تا اولین توکن): میانگین زمان انتظار کاربر برای دریافت اولین پاسخ، از 104.5 میلی‌ثانیه در بچ‌سایز ۱، به 13.39 ثانیه (معادل 13389.2 میلی‌ثانیه) در بچ‌سایز ۱۶ افزایش یافته است که ناشی از تشکیل صف طولانی‌تر در فاز پیش‌محاسبه (Prefill) است.
- شاخص TPOT (زمان تولید هر توکن خروجی): سرعت تولید توکن‌ها در مرحله دیکدینگ نیز تحت تأثیر بار پردازشی موازی روی کارت گرافیک قرار گرفته و از 17.7 میلی‌ثانیه به 122.4 میلی‌ثانیه رسیده است.
- صدک‌های پایداری (Percentiles مانند p95): بررسی صدک ۹۵ تأخیر کل نشان می‌دهد که در بچ‌سایز ۱۶، زمان انتظار درخواست‌های سنگین‌تر تا 26.4 ثانیه بالا رفته که نشان‌دهنده شکل‌گیری صف جدی در بارهای کاری سنگین است.

۲-۵ پایش منابع سخت‌افزاری و تحلیل رفتار سیستم (Hardware Metrics & Resource Utilization)

بررسی داده‌های استخراج شده از ابزارهای پایش سخت‌افزار در طول اجرای بنچمارک، زوایای پنهان‌تری از رفتار سیستم و محدودیت‌های گلوگاه (Bottleneck) سخت‌افزاری را آشکار می‌سازد:

- میزان مصرف حافظه ویدئویی (VRAM Utilization)

- وزن مدل (Model Weights): بیشترین حجم پایه VRAM بلافاصله پس از بارگذاری مدل اشغال شده و مقدار ثابت تخصیص یافته به پارامترهای کوانتیزه شده یا اصلی را شامل می شود.

- مدیریت حافظه KV Cache: در فاز Prefill با افزایش طول پرامپت ورودی و در فاز Decoding با رشد تعداد توکن های تولیدی، مصرف VRAM به صورت پویا افزایش می یابد. در بچ سایزهای بالا، تخصیص حافظه برای KV Cache به آستانه اشباع نزدیک شده و اهمیت مکانیزم های مدیریت حافظه (نظیر Paged Attention) را دوچندان می کند.

- پهنای باند و ترافیک باس (PCIe Bandwidth): میزان تبادل داده روی باس PCIe در طول اجرای استنتاج (پس از بارگذاری اولیه وزن ها در حافظه GPU) در سطح پایینی قرار دارد، چرا که محاسبات سنگین ماتریسی (Matrix Multiplication) مستقیماً درون واحدهای پردازشی شتاب دهنده انجام می شوند. با این حال، در سناریوهایی با جابجایی مکرر کانتکت یا ورودی/خروجی های حجیم، پایداری پهنای باند گذرگاه نقش مهمی در جلوگیری از ایجاد تاخیر ایفا می کند.

- بهره وری هسته های پردازشی و توان مصرفی (GPU Utilization & Power Draw):

- فاز Prefill: نمودار بهره وری هسته ها (Compute Units/SMs) نشان دهنده ی پالس های پردازشی بسیار سنگین و کوتاه مدت است که در آن کارت گرافیک به بالاترین حد توان پردازشی موازی (Compute-Bound) می رسد.

- فاز Decoding: رفتار پردازشی تغییر کرده و سیستم از حالت پردازشی به حالت Memory-Bound (محدود به پهنای باند حافظه دستگاه) منتقل می شود؛ در نتیجه، بهره وری لحظه ای هسته ها افت کرده ولی توان مصرفی (Power Draw) به دلیل فعالیت مداوم کنترلر حافظه در سطح پایداری باقی می ماند.

- پایداری حرارتی (Thermal Performance): بررسی دما در طول بنچمارک های طولانی مدت نشان داد که سیستم در محدوده ایمن عملکردی باقی مانده و افت فرکانس ناشی از حرارت بیش از حد (Thermal Throttling) گزارش نشده است که نشان دهنده پایداری سیستم خنک کننده در بارهای کاری مداوم است.

نتیجه نهایی (Trade-off Analysis): از دیدگاه مهندسی سیستم و معماری کامپیوتر، این آزمایش یک موازنه کلاسیک بین توان عملیاتی (Throughput) و تأخیر (Latency) را به نمایش می‌گذارد:

- **Batch size های پایین (مثل ۱):** مناسب برای کاربردهایی که پاسخگویی آنی (Real-time) و Low Latency حیاتی است، اما سخت‌افزار به شدت کم‌کار (Underutilized) می‌ماند.

- **Batch size های بالا (مثل ۱۶):** ایده‌آل برای سناریوهای پردازش دسته‌ای (Batch Processing) و سرویس‌دهی به تعداد زیادی از کاربران همزمان، چون حداکثر بهره‌وری و تعداد توکن بر ثانیه را به ارمغان می‌آورد؛ اما با افزایش چشمگیر تأخیر انتظار کاربر همراه است.

ریشه‌یابی سخت‌افزاری و پایداری سیستم: مشاهدات حاصل از پایش منابع نشان داد که مقیاس‌پذیری سیستم در بچ‌سایزهای بالا به شدت به مدیریت کارآمد VRAM و تخصیص حافظه برای KV Cache وابسته است. گذر از فاز Prefill (که سنگین و Compute-Bound است) به فاز Decoding (که حافظه‌محور است)، رفتار متفاوتی را به کنترلر حافظه و باس دستگاه تحمیل می‌کند. با این وجود، پایداری حرارتی مطلوب و عدم وقوع افت فرکانس ناشی از گرما (Thermal Throttling) گواهی می‌دهد که زیرساخت سخت‌افزاری پایداری لازم برای اجرای بارهای کاری مداوم و سنگین را داراست.

۶ منابع (لینک‌های مفید)

- CUDA Toolkit Documentation
<https://docs.nvidia.com/cuda/>
- CUDA C++ Programming Guide
<https://docs.nvidia.com/cuda/cuda-c-programming-guide/>
- CUDA Samples (VectorAdd, MatrixMul, etc.)
<https://github.com/NVIDIA/cuda-samples>
- NVIDIA Nsight Systems
<https://developer.nvidia.com/nsight-systems>

- NVIDIA Nsight Compute
<https://developer.nvidia.com/nsight-compute>
- NVIDIA System Management Interface (nvidia-smi)
<https://developer.nvidia.com/system-management-interface>
- NVIDIA CUDA Profiling Tools Interface (CUPTI)
<https://developer.nvidia.com/cupti>
- vLLM Documentation
<https://docs.vllm.ai/en/stable/>
- vLLM GitHub Repository
<https://github.com/vllm-project/vllm>
- Hugging Face Transformers
<https://huggingface.co/docs/transformers>
- GuideLLM Benchmark Tool
<https://github.com/vllm-project/guidellm>
- eBPF Official Website (Optional Monitoring Tool)
<https://ebpf.io/>
- NVIDIA Container Toolkit
<https://docs.nvidia.com/datacenter/cloud-native/container-toolkit/latest/install>
- Installing NVIDIA Drivers on Ubuntu 24.04
<https://linuxconfig.org/how-to-install-nvidia-drivers-on-ubuntu-24-04>
- FlashAttention Repository
<https://github.com/Dao-AI-Lab/flash-attention>
- LLM Inference Optimization (NVIDIA Developer Blog)
<https://developer.nvidia.com/blog/>